

IFR Protocol (Inferno / IFR)

Consolidated Audit Report

Complete audit series — seven documents, findings register CWA-01 ... CWA-82

Auditor	Collateral Web3 Open Audits
Client	NeaBouli / Inferno Protocol (ifrunit.tech)
Date	2026-09-14
Baselines	repo NeaBouli/inferno @ eb538a35 / e8cf1ece · Ethereum mainnet blocks 25971217 / 25974493 · live web surfaces 2026-09-13/14
Method skills	collateral-web3-audit · web3-contract-deep-audit · web3-integration-audit · web3-content-coherence-audit · ai-readiness-audit

Overall result

82 findings (CWA-01 ... CWA-82): 0 Critical · 3 High · 29 Medium · 34 Low · 16 Informational. Of the 3 High findings, one is dormant (verified fail-closed on-chain) and two are conditional on the creator-gateway service being publicly deployed (deployment status not observable read-only). Severity totals are aggregated from the per-document summaries below; the per-document sections are authoritative.

Contents

Document	Register	Findings
Part 1 — Full-Scope Security Audit	CWA-01 ... CWA-24	0 C · 1 H (dormant) · 6 M · 12 L · informational cluster
Part 2 — Contract Deep Security Audit	CWA-25 ... CWA-27	0 C · 0 H · 1 M · 1 L · 1 I
Part 3 — Surfaces Supplement (apps & backends)	CWA-28 ... CWA-45	0 C · 2 H · 5 M · 8 L · 3 I

Part 4 — Web3 Integration Audit (wallets & wiring)	CWA-46 ... CWA-50	0 C · 0 H · 1 M · 2 L · 2 I
Part 5 — README Audit (claim-by-claim)	CWA-51 ... CWA-56	0 C · 0 H · 2 M · 1 L · 3 I
Part 6 — Content Coherence Audit (landing + wiki + shop)	CWA-57 ... CWA-74	0 C · 0 H · 10 M · 7 L · 1 I
Part 7 — AI Readiness Audit (copilot + AI anchors)	CWA-75 ... CWA-82	0 C · 0 H · 4 M · 3 L · 1 I

Editorial correction, 2026-09-19: the consolidated severity totals and contents table now follow the 82 individual finding labels. The original generated PDF counted CWA-64 as Medium instead of its published Low label and counted two informational findings for the three-item CWA-25...27 register. No finding text, severity label or technical conclusion was changed.

Each part is reproduced in full and remains independently authoritative, with its own baselines, evidence references and SHA-256 pins registered in `docs/community-audits/README.md` of the repository.

IFR Protocol (Inferno / IFR) — Security Audit Report

Auditor: Collateral Web3 Open Audits **Client:** NeaBouli / Inferno Protocol (ifrunit.tech) **Date:** 2026-09-14 (evidence gathered 2026-09-13 21:29–22:30 UTC) **Scope baseline:** repo NeaBouli/inferno @ commit eb538a355001b042b343bfb19221af2407a96e63 · Ethereum mainnet block 25971217 (0x18c4a11, pinned 2026-09-13T21:29:57Z via two independent RPCs) · web surfaces as of 2026-09-13 **Report version:** 1.0

1. Executive Summary

IFR Protocol is a deflationary ERC-20 ecosystem with 14 deployed mainnet contracts, a static website + wiki (36 pages), a wallet dApp, a Next.js shop PWA, and three backend APIs. This audit performed a full manual review of all 14 deployed contract sources at the pinned commit, reproduced the complete test suite (644 contract + 30 builder + 36 SDK tests, all passing), verified the security-relevant on-chain state at block 25971217 with direct RPC reads, and checked the web/API surface read-only.

The codebase is in good shape: modern Solidity (0.8.20, OpenZeppelin v5), consistent Checks- Effects- Interactions or guarded patterns, no delegatecall/selfdestruct/assembly anywhere, bounded parameters nearly everywhere, strong CI hygiene (SHA-pinned actions, minimal workflow permissions, green Slither/Mythril/gitleaks gates), and a mature known-issues culture whose register we verified item by item. No Critical issues were found.

The single most important finding is **dormant**: LendingVault's entire collateral math depends on one governance-set price (`ifrPriceWei`). It is currently 0 (borrowing fail-closed — verified on-chain), but any future activation without a bounded/TWAP oracle would allow a single timelocked proposal to drain all 52.16M IFR held in the vault with dust collateral. Activation must be gated on the remediation in CWA-01.

The single most important *new* finding is that the documented fee flywheel is broken in practice: 371,543 IFR of pool fees are permanently stranded in FeeRouterV1, which has no token withdrawal function (CWA-02). The strongest operational strengths: the 3-of-5 Safe ownership of governance (verified on-chain for all three Safes), the genuinely irreversible LP lock inside BootstrapVaultV3 (verified: vault holds all LP except the 1000-wei minimum, no withdrawal function exists), and the reproducible test/CI evidence.

Bottom line: shippable in its current fail-closed configuration, subject to the Medium findings. Do **not** activate lending or price-conditioned commitments before CWA-01/CWA-03 remediations. Fix the documentation drift (§6) — several public claims no longer match chain state in both directions (overstatements *and* unreported improvements).

Verdict overview:

Area	Result	Findings
Smart contracts (14 mainnet)	PASS w/ findings	C:0 H:1* M:3 L:6 I:2 (*H dormant)
On-chain state & roles	PASS w/ findings	M:2 L:1
Web surfaces (apex, web3, shop, wiki)	PASS w/ findings	L:1 I:2
APIs (copilot, points, verify, shop)	PASS w/ findings	M:3 L:3
Repository & supply chain	PASS w/ findings	M:1 I:2

Area	Result	Findings
Documentation & claims consistency	WARN	I: cluster (11 entries)
Governance & centralization	PASS w/ findings	covered above

Totals: **0 Critical** · **1 High (dormant)** · **6 Medium** · **12 Low** · **Informational cluster**

2. Scope and Methodology

In scope:

- All 14 deployed mainnet contracts (full manual source review at `eb538a35`): InfernoToken, Governance, IFRLock, Vesting, LiquidityReserve, BuybackVault, BurnReserve, PartnerVault, FeeRouterV1, BootstrapVaultV3, BuilderRegistry, CommitmentVault, LendingVault, BuybackController — plus the Uniswap V2 LP pair and 3 Gnosis Safes (on-chain state).
- On-chain state at block 25971217: ~80 `cast call` reads (roles, parameters, feeExempt map, balances, Safe thresholds/owners, proposals #15/#16, C2 tranches, LP custody).
- Repo hygiene: branch protection, 16 CI workflows (pinning, permissions, run history), secret sweep, `npm audit`, full local test run.
- Web surfaces: header matrix on 6 hosts, address-integrity extraction from `homepage/ /web3/ (apex+subdomain)/ builder.html`, XSS-sink inspection, SRI check.
- APIs: copilot-api (full `server/index.ts` review, 1,966 lines), points-backend (auth, voucher, rate-limit, lockProof, voucher-signer), live GET endpoint re-checks.
- Claims register verification (§6).

Out of scope: Sepolia deployments beyond spot-checks; `patches/` historical archive (299 diffs); `apps/` sub-apps other than the three named backends (dashboard, governance-dashboard, creator-gateway, telegram bot, SDK, wallet prototype — CI-green at HEAD but not manually reviewed); benefits-network route-level business logic (middleware/config level only); private infrastructure (Hetzner hosts, Railway project, Telegram bot runtime); third-party services (Uniswap, Etherscan, GitHub Pages).

Methods: manual line-by-line source review at the pinned commit; read-only on-chain calls (`cast 1.7.1`, `drpc/publicnode`, block-pinned); local reproduction of the full test suite (`npm ci + npm run test:contracts:all`, exit 0); public GET/HEAD observation of live surfaces; GitHub API metadata inspection; documentation cross-consistency analysis.

Limitations: Slither/Mythril were **not re-run locally** (toolchain not installed); their evidence is CI-attested (green runs at/around the pinned commit — see §9) plus baseline file review. Etherscan verification was spot-checked on InfernoToken only (verified source: v0.8.20+commit.a1b79de6, paris, no optimization); the remaining "16/16 verified" claim rests on project docs + CI. No active probing, no authenticated API tests, no transactions, no POSTs to live services. The Gitleaks scan is CI-attested (green at HEAD) plus a local pattern sweep (gitleaks binary not installed locally).

3. System Overview

IFR is a 1,000,000,000-supply ERC-20 (9 decimals) with fee-on-transfer: 2.0% sender burn + 0.5% recipient burn + 1.0% pool fee, hard-capped at 5% total. All protocol parameters are owned by a custom Governance timelock (48h delay, verified on-chain), whose owner is the Treasury Gnosis Safe (3-of-5, verified). Safes are the only proxy-capable components; all 14 protocol contracts are immutable, non-upgradeable, no-proxy.

Access/gating: `IFRLock` (simple accumulating lock; unlock always available, even when paused — source-verified), `CommitmentVault` (4 condition types; price paths currently fail-closed), `Vesting` (team 150M, cliff ends ~2027-03-05). Liquidity: `LiquidityReserve` (200M, 180d lock ended 2026-09-01, 50M/90d staged withdrawals — live but unused), `BootstrapVaultV3` (finalized; LP permanently held by the vault itself). Ecosystem: `PartnerVault` (40M, bounded rewards), `FeeRouterV1` (0.05% ETH swap fee, EIP-712 vouchers), `BuilderRegistry` (0 builders), `BuybackVault` / `BuybackController` (ETH→IFR buyback; both currently unfunded, 0 ETH at baseline). Web: GitHub Pages apex + 36-page wiki, nginx-hosted `web3` dApp, Next.js shop, Express APIs (`copilot-api`, `points-api`, `verify-api`).

Roles at block 25971217 (all chain-verified):

Role	Holder	Powers	Delay/Threshold
Governance owner	Treasury Safe <code>0x5ad6...cE3b</code>	propose/execute/cancel timelock	3-of-5
Governance guardian	Deployer EOA <code>0x6b36...ed67</code>	cancel proposals only	none (EOA)
Timelock delay	—	172,800 s (48h)	MIN 1h / MAX 30d (onlySelf)
All contract owners/admins	Governance <code>0xc43d...D041</code>	parameters, feeExempt, price, adapters	48h timelock
Contract guardians (IFRLock, Vesting, LiquidityReserve, PartnerVault, BurnReserve, BuybackVault+Controller)	Deployer EOA <code>0x6b36...ed67</code>	pause only (burn also for BurnReserve)	none (EOA)
FeeRouter voucherSigner	<code>0x17F8...2d4</code> (= Safe owner M.G.)	sign EIP-712 discount vouchers	hot key
Vesting beneficiary	<code>0x04FA...6239</code> (immutable)	claim vested team tokens	none

4. Findings

Critical

None.

High

[CWA-01] LendingVault activation risk: single governance-set price controls all collateral math

- **Severity:** High (dormant) · **Likelihood:** — (currently impossible; activates on price set) · **Status:** Accepted risk — verified fail-closed
- **Component:** `LendingVault.sol` (`0x9743...EB9DF`) — `setIFRPrice()`, `getRequiredCollateral()`, `getCollateralRatio()`
- **Cross-refs:** OPS-001; extends OKComputer "manual-oracle risk"
- **Description:** `ifrPriceWei` is set by a plain `onlyOwner` call with no bounds, no freshness, no oracle. Every borrow and every liquidation derives from this single number. **Verified closed:** `ifrPriceWei() = 0` at block 25971217, and `getRequiredCollateral` reverts "price not set" while zero — borrowing is impossible today.

- **Impact / exploit path (post-activation):** after any executed proposal `setIFRPrice(X)` :
 1. If X is set far below market: `requiredCollateral = amount * X * 200 / 1e11` collapses — borrowing the entire `totalAvailable` (52,155,440.95 IFR, verified) costs dust ETH (at X = 1 wei: ≈ 104,310,881 wei of collateral — $52,155,440.952845656e9 \times 200 / 1e11$ — still dust against any real reserve). One transaction drains the vault.
 2. If X is set far above market: every existing loan's collateral ratio collapses → permissionless `liquidate()` hands all borrower collateral to liquidator (5%) and lender. Mitigations today: 48h timelock (public visibility), guardian cancel, the 4-hour CI invariant monitor. None of these *prevent* execution — they only allow reaction.
- **Evidence:** source `LendingVault.sol:350–355, 358–367, 395–399` @ `eb538a35` ; chain reads block 25971217 (`ifrPriceWei() = 0`, `totalAvailable() = 52,155,440.952845656e9`).
- **Recommendation:** before any activation: (a) bound `setIFRPrice` to a sanity band or replace with a Uniswap TWAP read (the repo already documents this as Phase-2 requirement); (b) add a per-tx borrow cap and per-price-change rate limit (e.g. max ±20% per update, min 24h between updates); (c) set `protocolFeeReceiver` (CWA-08) and document the activation checklist as a governance-gated runbook. Until then, keep `ifrPriceWei = 0`.

Medium

[CWA-02] 371,543 IFR of pool fees permanently stranded in FeeRouterV1 — documented flywheel cannot happen

- **Severity:** Medium · **Likelihood:** certain (already the case) · **Status:** Open — new finding
- **Component:** `FeeRouterV1.sol` (`0x4807...667C7`), token `poolFeeReceiver`
- **Description:** InfernoToken routes the 1% pool fee (in IFR) to `poolFeeReceiver`, which is `FeeRouterV1` (Proposal #6). `FeeRouterV1` has **no token withdrawal/sweep/rescue function** (full source reviewed; `grep` for `withdraw|sweep|rescue|recover` finds only ECDSA `recovered` variables). Its only outward value flow is ETH (`feeCollector.call{value:}`). IFR tokens therefore accumulate with no extraction path — not even governance can move them.
- **Impact:** 371,543.991017522 IFR (≈ 0.037% of supply) are unrecoverable at the baseline, and every future non-exempt transfer grows the sink. Worse, the docs describe a live flywheel: `FEE_DESIGN.md` claims pool fees "flow directly into the 50/50 buyback+burn / LP-deepening flywheel"; `transparency.html` states "all pool fees route correctly to `FeeRouterV1` → `BuybackVault` + `BurnReserve`"; `press-kit` repeats it. That flow does not exist for the IFR-denominated pool fee — only for ETH swap fees.
- **Evidence:** chain reads block 25971217 (`poolFeeReceiver() = 0x4807...667C7`, `balanceOf(FeeRouterV1) = 371,543.991...e9`); source `FeeRouterV1.sol` (full read); `docs/FEE_DESIGN.md:63`, `docs/wiki/transparency.html:1365`, `docs/wiki/press-kit.html:709`.
- **Recommendation:** either (a) accept and document `FeeRouterV1` as a permanent IFR sink (economically a slow burn — but it does *not* reduce `totalSupply`), correcting the three doc locations; or (b) route `poolFeeReceiver` to a contract with a governed sweep (e.g. `BuybackController`, which has `withdrawIFR`), via timelock proposal. Never deploy a value-holding contract without an extraction path.

[CWA-03] CommitmentVault: price-conditioned tranches lock funds permanently — docs understate as "fail-closed"

- **Severity:** Medium · **Likelihood:** medium (permissionless, reachable via raw contract or any UI exposing the type) · **Status:** Open — extends OPS-002
- **Component:** `CommitmentVault.sol` (`0x0719...73d3`) — `_getCurrentPrice()`, `isConditionMet()`

- **Description:** `_getCurrentPrice()` is a stub that **always returns 0** (the TWAP call is commented out), and `isConditionMet` additionally requires `priceOracle != 0`. Setting the oracle later via `setPriceOracle` changes nothing — price can never be "met". For `PRICE_ONLY` and `TIME_AND_PRICE` tranches this means **no unlock path exists at all**: owner-unlock requires the condition, and the 30-day permissionless auto-unlock also requires the condition. There is no admin rescue. On-chain, C2's 10 tranches are all `TIME_ONLY` (verified tranche 0: `cType=0`, `unlockTime=1785283200` = 2026-07-29 ✓), so no funds are currently affected — but any user can `lock()` a price-conditioned tranche today.
- **Impact:** irreversible loss of the locked amount for the user; reputational damage framed as "the contract ate my tokens". Current exposure: 47,952,476.87 IFR total locked (all time-conditioned, safe).
- **Evidence:** source `CommitmentVault.sol:124-142, 164-186, 238-243` @ eb538a35 ; chain reads block 25971217 (`priceOracle() = 0x0`, `p0() = 300,000,000`, C2 tranche probe).
- **Recommendation:** until a real oracle ships, revert on lock attempts with `cType != TIME_ONLY` (one-line guard), or add an owner-governed rescue for price-tranches after a long grace period. Update wiki wording from "fail-closed" to "price locks are currently unavailable; do not create them — funds cannot be recovered."

[CWA-04] points-backend: SIWE verification lacks domain binding (cross-domain replay)

- **Severity:** Medium · **Likelihood:** medium (requires victim to sign on a malicious site) · **Status:** Open — new finding
- **Component:** `apps/points-backend/src/routes/auth.ts:38`
- **Description:** `siweMessage.verify({ signature })` is called **without** `{ domain, nonce }` expectations. The code separately checks the nonce against its own store (good), but never verifies that the signed message's domain is `points-api.ifrunit.tech` (or the intended origin). A phishing page can request a nonce from the public `/auth/siwe/nonce` endpoint, ask the victim to "Sign in with Ethereum" on the attacker's domain, and replay the signature to `/auth/siwe/verify` to obtain a 24h JWT for the victim's wallet.
- **Impact:** impersonation within the points/voucher system under the victim's wallet: posting points events, and — if the victim also satisfies the lock proof (CWA-05 caveats) — claiming the daily voucher. No on-chain fund movement is possible with the JWT alone, which caps the severity.
- **Evidence:** `auth.ts:37-51` @ eb538a35 (no domain check); JWT issuance `middleware/auth.ts:14-20` (HS256, 24h — matches docs).
- **Recommendation:** pass `{ domain: <expected>, nonce: <from store> }` into `siweMessage.verify()` and reject mismatches; also bind `uri/origin` if multiple frontends are intended. This is the standard SIWE hardening step and costs two lines.

[CWA-05] points-backend lock proof defaults to Sepolia IFRLock — mainnet gating depends on unset env vars

- **Severity:** Medium · **Likelihood:** unknown (production env not observable read-only) · **Status:** Open — config-dependent
- **Component:** `apps/points-backend/src/middleware/lockProof.ts:5-6`
- **Description:** `RPC_URL` defaults to `https://rpc.sepolia.org` and `IFR_LOCK_ADDRESS` defaults to the Sepolia IFRLock (`0x0Cab...75d3`). If the production deployment of points-api does not explicitly override both, the voucher-issuing lock proof checks testnet locks — and Sepolia IFR is free, so the "min 1,000 IFR locked" Sybil barrier collapses.

- **Impact:** bypass of the voucher lock requirement → unlimited-fee-discount eligibility (bounded per-wallet/day and by the global daily cap) without real mainnet commitment. No direct fund risk; the 15-bps voucher discount is economically small. The positive parts: `SKIP_LOCK_PR00F` is correctly disabled in production, RPC failure fails closed (503), and results are cached 5 min.
- **Evidence:** `lockProof.ts:5-13,23-66 @ eb538a35`; mainnet IFRLock is `0x7699...54Eb` (chain-verified).
- **Recommendation:** production-throw on missing `IFR_LOCK_ADDRESS` /mainnet RPC (same pattern already used for `JWT_SECRET` in `auth.ts:5`), and add a startup log line asserting `chainId=1`.

[CWA-06] Voucher signer key doubles as Gnosis Safe owner key (hot-key/multisig overlap)

- **Severity:** Medium · **Likelihood:** low-medium (requires server/key compromise) · **Status:** Open — verified on-chain overlap
- **Component:** `FeeRouterV1 voucherSigner` vs. Safe owner sets
- **Description:** chain reads show `FeeRouterV1.voucherSigner() = 0x17F8DD6dECCb3ff5d95691982B85A87d7d9872d4` — the **same address** is an owner of all three 3-of-5 Safes (`getOwners()` verified on all three). The voucher signing key must be available to the points-backend at runtime (`VOUCHER_SIGNER_PRIVATE_KEY` env, `voucher-signer.ts:35-39`). If that env holds the key for `0x17F8...2d4`, a single backend compromise yields one of the three keys needed to move the protocol Safes (400.6M IFR in LP Reserve Safe alone) — and erodes the multisig's independence assumption (one person/key, two security domains).
- **Evidence:** chain reads block 25971217 (`voucherSigner()`, `getOwners() x3`); `apps/points-backend/src/services/voucher-signer.ts @ eb538a35`; the project transparency page itself labels `0x17F8...` as both "Signer M.G." and "Voucher Signer".
- **Recommendation:** rotate `voucherSigner` to a dedicated hot key that owns nothing else (`setVoucherSigner` via timelock), and state publicly that no Safe owner key is ever stored on a server. If the deployed key is already distinct from the Safe key (i.e. the on-chain `voucherSigner` address is operated differently than assumed), document the key-management separation in `OFFCHAIN_SECURITY.md`.

[CWA-07] Branch protection on `main` does not require the CI gates the project advertises

- **Severity:** Medium · **Likelihood:** n/a (process gap) · **Status:** Open
- **Component:** GitHub repo settings (`gh api repos/NeaBouli/inferno/branches/main/protection`)
- **Description:** protection requires 1 approving review + linear history and blocks force-pushes, but **required_status_checks is null and enforce_admins is false**. Every gate the security story leans on — Slither baseline, Mythril, contract tests, gitleaks, dependency audit — can be bypassed by an admin pushing straight to main or merging a red PR. The gates run and are green (verified), but they are advisory, not enforced.
- **Impact:** one compromised or careless admin action silently lands un-gated code on main, and the public "0 unreviewed High / CI-gated" claims would no longer hold for that code.
- **Evidence:** API response captured in `evidence/repo-ci.txt` (2026-09-13).
- **Recommendation:** require the `contracts.yml`, `security-audit.yml` (and ideally `mythril-analysis.yml`) checks on main, and enable `enforce_admins`.

Low

[CWA-08] LendingVault: pre-activation configuration gaps (protocolFeeReceiver unset; 2300-gas collateral returns; no loan-expiry enforcement)

- **Severity:** Low (dormant) · **Likelihood:** — · **Status:** Open
- **Component:** LendingVault.sol (repay() :245-247, :255; liquidate() :297-299; no expiry path)
- **Description:** (a) protocolFeeReceiver() = 0x0 at baseline — once loans repay with the receiver unset, the protocol's 50% interest share accumulates inside the vault with **no sweep function** (same anti-pattern class as CWA-02). (b) Collateral returns use payable(x).transfer() (2300 gas): a borrower whose address is a contract with a gas-hungry/reverting receive() can never repay() (collateral frozen), and a lender with such an address makes its loans **un-liquidatable** (liquidate() reverts on the lender transfer). (c) Loans past duration accrue at the same rate with no default/liquidation trigger — expiry is not enforced.
- **Recommendation:** (a) set protocolFeeReceiver in the same activation proposal as the price (or add a governed sweep); (b) replace .transfer with .call + reentrancy guard (already present) or pull-payments; (c) decide and document the intended overdue-loan path before activation (e.g. liquidatable after startTime + duration + grace regardless of ratio).

[CWA-09] Guardian powers concentrated in a single EOA across all contracts

- **Severity:** Low · **Likelihood:** low · **Status:** Acknowledged by project (Guardian-multisig migration listed as Pending in mainnet-checklist) — verified still EOA everywhere
- **Component:** Governance guardian, IFRLock/Vesting/LiquidityReserve/PartnerVault/BurnReserve/BuybackVault/BuybackController guardians — all 0x6b36...ed67 (block 25971217)
- **Description:** the Deployer EOA can cancel any timelock proposal and pause new locks, vesting releases, reserve withdrawals, partner claims and buybacks. It cannot move funds, but a compromised key = indefinite protocol-wide griefing. Additionally Governance.setGuardian() is onlyOwner but **not routed through the timelock** (W15, confirmed at Governance.sol:120-125) — the 3-of-5 Safe can swap the guardian instantly; symmetric, but worth a line in the trust model.
- **Recommendation:** complete the planned guardian-multisig migration; meanwhile publish the guardian key-storage policy. Timelock the guardian change or document why instant rotation is deliberate (rapid response cuts both ways).

[CWA-10] web3.ifrunit.tech serves the wallet dApp with no hardening headers

- **Severity:** Low · **Likelihood:** low · **Status:** Open (re-verified this run)
- **Component:** web3.ifrunit.tech (nginx) — and, immutably, GitHub Pages apex
- **Description:** no CSP, no frame protection, no HSTS, no nosniff on the host that embeds the wallet-connect and lock/unlock UI (re-checked 2026-09-13: only cache-control: no-store). The shop subdomain shows the project can do this well (full CSP, XFO DENY, HSTS 2y, Referrer/Permissions-Policy). Clickjacking the dApp is mitigated by wallet-native confirmation dialogs but remains a real phishing-adjacent risk. GitHub Pages cannot set custom headers — the nginx host can.
- **Recommendation:** ship the shop's header set on web3.ifrunit.tech (nginx config), at minimum frame-ancestors 'none' + HSTS + nosniff; consider a long-term note that the apex surface is header-limited by the Pages platform.

[CWA-11] copilot-api /api/health discloses backend configuration

- **Severity:** Low · **Likelihood:** n/a · **Status:** Open (observed live)
- **Component:** apps/ai-copilot/server/index.ts:546-553
- **Description:** the public health endpoint reveals `apiKeySet:true`, `etherscanKeySet:true` and a deploy version string. Useful to an attacker mapping backend capabilities; zero direct impact. Also observed: `/api/ifr/price` returning 502 at audit time (upstream RPC/Etherscan failure mode — correctly errors rather than serving stale data; the frontend honestly displays "pool data could not be verified" — good fail-closed behavior).
- **Recommendation:** reduce `/api/health` to `{"status":"ok"}` publicly; keep key-state diagnostics on an internal/flagged endpoint.

[CWA-12] copilot-api: LLM cost guard is warn-only; per-IP limits don't bound aggregate spend

- **Severity:** Low · **Likelihood:** medium under targeted abuse · **Status:** Open
- **Component:** apps/ai-copilot/server/index.ts:62-78 + rate limiter :23-60
- **Description:** 5 msg/min + 20 msg/hour per IP, in-memory; the \$1/day "cost warning" only logs. A distributed abuser (many IPs) can drive Anthropic spend without hitting any per-IP limit. In-memory buckets also reset on every deploy/restart.
- **Recommendation:** add a hard global daily cutoff (429 once the budget is spent), and/or a global concurrent-request cap; keep per-IP limits as-is.

[CWA-13] /api/bootstrap/vote accepts votes for any contributor wallet without a signature

- **Severity:** Low · **Likelihood:** — (bootstrap concluded) · **Status:** Open (live code path)
- **Component:** apps/ai-copilot/server/index.ts:1202-1226
- **Description:** the endpoint checks the wallet's on-chain bootstrap contribution (good) but requires no proof of key control — anyone can cast/flip a vote as any contributor. The poll is historical (vault finalized, verified), so impact is nil today; the pattern is live code and could be copied into a future vote where it would matter.
- **Recommendation:** require an EIP-191 signature over the vote, or retire the endpoint.

[CWA-14] points-backend: voucher issuance doesn't redeem points; /voucher/validate/:nonce exposes the voucher's wallet

- **Severity:** Low · **Likelihood:** low · **Status:** Open
- **Component:** apps/points-backend/src/routes/voucher.ts:23,83-116
- **Description:** (a) the 100-point threshold is a gate, not a payment — points are never decremented, so 100 points = one voucher per day forever (bounded by the daily per-wallet and global caps; possibly intended — docs read like redemption). (b) The public validate endpoint returns the owning wallet address for any known nonce; nonces are 256-bit random so this is not enumerable, but a leaked nonce (e.g. from mempool/tx data when used on-chain) deanonymizes the linkage.
- **Recommendation:** confirm intended economics (gate vs. redeem) and document; omit the wallet from the public validate response (or hash it).

[CWA-15] LiquidityReserve: withdrawal cap is governance-mutable, not an invariant

- **Severity:** Low · **Likelihood:** low · **Status:** Open

- **Component:** `LiquidityReserve.sol:125–129`
- **Description:** the publicly communicated "50M IFR per 90 days" safety cap is a `onlyOwner` setter away from any value (via 48h timelock). Verified current state: `maxWithdrawPerPeriod = 50M`, `totalWithdrawn = 0`, lock ended 2026-09-01 (`lockEnd = 1788222659`). The timelock makes changes visible; just don't present the cap as hardcoded.
- **Recommendation:** docs should say "cap changeable via 48h governance"; optionally emit an off-chain alert hook on `MaxWithdrawPerPeriodUpdated`.

[CWA-16] Single-step `transferOwnership` across reserve contracts

- **Severity:** Low · **Likelihood:** low · **Status:** Open
- **Component:** `LiquidityReserve.sol:155–160`, `BurnReserve.sol:96–101`, `BuybackVault.sol:165–172`, `BuybackController.sol:249–254`
- **Description:** one transaction to a mistyped address bricks the contract's administration (`owner=0` checks exist, but no two-step accept). Current owner is Governance, so a transfer itself requires the timelock — residual risk is a bad data payload, which the 48h window and guardian cancel mitigate.
- **Recommendation:** adopt OZ `ownable2step` in any future contract version.

[CWA-17] LP Reserve Safe is not `feeExempt` — LP operations from it pay ~3.5%

- **Severity:** Low · **Likelihood:** certain when used · **Status:** Open — verified on-chain
- **Component:** token `feeExempt` map vs. LP Reserve Safe `0x5D93...C04` (holds 400.6M IFR)
- **Description:** `feeExempt[LPReserveSafe] = false` (block 25971217) while the other two Safes are exempt. Any LP-deepening transfer from this Safe pays the full transfer fee (50M $\Rightarrow$ ~1.75M IFR burned+fees). This may be deliberate (deflationary on ops moves) — it is not documented either way, and the fee-exempt table in the wiki predates these entries.
- **Recommendation:** decide deliberately; if exempted, note that the Safe then also skips the *burn*, changing the deflation accounting for large moves.

[CWA-18] `FeeRouterV1`: zero-address setters can burn user fees; voucher `maxUses` is dead code

- **Severity:** Low · **Likelihood:** low (requires governance error) · **Status:** Known (W10, W19) — source-verified
- **Component:** `FeeRouterV1.sol:208–223` (no zero checks on `setVoucherSigner` / `setFeeCollector`), `:42–48, 123–155` (`maxUses` never enforced)
- **Description:** (a) if governance ever sets `feeCollector = 0x0`, the fee `.call{value:}` to `address(0)` **succeeds** and burns users' ETH fees silently; (b) `maxUses` is signed and stored but never checked — practically moot because the nonce mechanism already enforces single-use, but the field misleads integrators reading the typehash.
- **Recommendation:** add `!= address(0)` requires (one line each); drop `maxUses` from a V2 typehash or enforce it.

[CWA-19] Dormant design notes on Buyback path (no `nonReentrant`; unchecked transfer; MEV window)

- **Severity:** Low · **Likelihood:** low while unfunded · **Status:** Known (W3/W4/W11-class + Slither baseline entries) — source-verified
- **Component:** `BuybackVault.sol:105–137`, `BuybackController.sol:128–147, 243–247`

- **Description:** neither buyback executor has a reentrancy guard (external calls to a governance-set router; bounded by cooldown/owner gates); `BuybackController.withdrawIFR` ignores the ERC20 return value (safe with the known IFR token, fragile as a pattern); permissionless `execute()` with 5% slippage on a thin pool is sandwichable. All currently moot: both contracts hold 0 ETH / 0 IFR at baseline (verified).
- **Recommendation:** keep the Slither baseline entries; add `nonReentrant` + return checks in the next contract version; don't fund either contract before that.

Informational

- **[CWA-20] Documentation drift cluster** (details in §6): test counts 642 (homepage) vs 644 (actual, wiki) vs 544 (copilot dev-mode welcome, live in production); four divergent tier tables across `wiki/integration/press-kit/API (/api/ifr/check` uses 500/2,000/10,000 — a fourth variant); `rewardBps` 1000 (docs) vs 1500 (chain); `PartnerVault` throttle described as active but `ifrLock = 0x0` on-chain; `Community Safe` 57.9M (docs) vs 7.9M (chain — ~50M moved, consistent with LP-strategy milestones but not reflected in the pages reviewed); LP phase figures 150/100/150M vs 100/100/100M; signer #1 named "A.K." vs "G.M. (Kaspartizan)"; vesting wiki `start: 1741168424` (a 2025 date, impossible) vs on-chain `1772670647` (= 2026-03-05T00:27 UTC, matching genesis); `DEPLOYMENTS.md` lists `BuybackController` owner = `Deployer EOA` while chain shows `Governance` (an *unreported improvement*); wiki fee-exempt table (10 entries) vs chain (15 exempt among audited set, incl. Vesting — resolving W6 in practice); Etherscan token info text still says "Community Bootstrap active".
- **[CWA-21] License inconsistency:** all contracts carry `SPDX-License-Identifier: MIT`; the README states "© 2026 Inferno Protocol. All rights reserved."; no `LICENSE` file exists. Pick one and align all three locations.
- **[CWA-22] Process surface:** no `security.txt` on any host; security contact = GitHub Private Vulnerability Reporting only; no bug bounty (planned Phase 5); three dependabot PRs open at audit time (#81–#83, non-security version bumps); Etherscan "16/16 verified" spot-confirmed for the token only.
- **[CWA-23] `deployments/mainnet.json` records only `BuybackController`** — tooling trusting that file sees an incomplete deployment truth; authoritative mapping lives in docs.
- **[CWA-24] Copilot trust chain:** the copilot server fetches live wiki HTML hourly and injects it into the LLM system context (`index.ts:90–159`) — a GitHub Pages content compromise translates into copilot answers. Accepted as design; worth a sentence in the threat model.

5. Known-Issues Verification

Every register entry was verified against source @ `eb538a35` and chain state @ block 25971217:

Item	Documented status	Audit verification
W1 <code>setOwner()</code> timelock bypass	fixed (onlySelf via timelock)	Confirmed fixed — <code>Governance.sol:128–133</code> ; owner = Safe since 20.03.2026
W3 unbounded <code>setParams</code> (<code>BuybackVault</code>)	partially addressed	Confirmed — bounds present ($\leq 100\%$, non-zero addresses); cooldown unbounded (owner-only)
W4 MEV on buyback/bootstrap	accepted	Confirmed open , dormant (contracts unfunded); see CWA-19
W6 Vesting releases pay 3.5% fees	accepted	Superseded on-chain: <code>feeExempt[Vesting] = true</code> — docs stale (CWA-20)

Item	Documented status	Audit verification
W10 FeeRouter zero-address setters	open	Confirmed open (CWA-18)
W11 FeeRouter/Buyback no nonReentrant	open	Confirmed open , bounded (CWA-18/19)
W13 bootstrap 0-min LP add	accepted	Confirmed — one-time, executed 05.06.2026
W15 <code>setGuardian</code> not timelocked	open	Confirmed open (CWA-09)
W17 LP stranded in BootstrapVaultV3	accepted	Verified on-chain: <code>lpLockId = 0</code> (Team.Finance unused), vault holds 54,772,255.750515611 of 54,772,255.750516611 LP (rest = 1000-wei Uniswap minimum); no withdrawal function exists. This is a <i>feature</i> (permanent lock) — correctly documented
W18 Vesting guardian not rotatable	open	Outdated: <code>transferGuardian(address)</code> exists on-chain (selector probe reverts <code>OnlyGuardian()</code> for strangers) and in source
W19 voucher <code>maxUses</code> unenforced	open	Confirmed open , impact reframed: nonce single-use dominates (CWA-18)
OPS-001 lending fail-closed (<code>ifrPriceWei = 0</code>)	active	Verified on-chain (CWA-01)
OPS-002 commitment price fail-closed (<code>priceOracle = 0x0</code>)	active	Verified on-chain ; extended by CWA-03 (permanence not documented)
OPS-005 BuilderRegistry empty	active	Verified on-chain: <code>getActiveBuilders() = []</code>
Slither baseline (6 reviewed High)	CI-gated	Confirmed present in repo; CI gate green at HEAD; not re-run locally (limitation §2)
Mythril bounded runs (0 signals claimed)	CI-gated	CI history green (latest 2026-09-12 push: success); not re-run locally
~21,000 IFR legacy pool fees on Deployer	accepted, "not re-routed"	Resolved on-chain: Deployer IFR balance = 0 at baseline; docs' decision note remains historically accurate

6. Documentation & Claims Consistency

Register results (claim → verdict → evidence):

Claim	Verdict	Evidence
644 contract tests / 30 builder / 36 SDK pass	Confirmed	local run at <code>eb538a35</code> : 644 passing (22s) + 30 passing + 36 passing, exit 0 (evidence/test-run.log)
Homepage "642 tests"	Diverged (stale, under-reports)	actual 644
Copilot dev-mode "544 documented tests"	Diverged (stale)	live widget string vs. 644+30+36
Coverage 99% stmt / 91% branch	Unverifiable as current	historical snapshot (05.03.2026) only; no current coverage run in CI

Claim	Verdict	Evidence
"17 on-chain components"	Plausible, definition-dependent	14 protocol contracts + LP pair + ...; recommend publishing the counted list
Etherscan "16/16 verified"	Spot-confirmed	token page shows verified source (v0.8.20 paris); full set not individually fetched
Supply 997,571,140.02 / burned 2,428,859.98	Confirmed exactly	chain <code>totalSupply()</code> = 997,571,140.022456196 IFR @ 25971217; API matches to the unit
LiquidityReserve 200M / 0 withdrawn / 50M-90d cap	Confirmed	chain reads; <code>totalWithdrawn</code> = 0
Vault invariant snapshot (CV 47,952,476.87 / LV 52,155,440.95, 0 lent)	Confirmed exactly	chain balances + <code>getOfferCount()</code> =3, <code>getLoanCount()</code> =0
C2 lock: 10 tranches, 20,156,940.95 IFR, unlock 29.07.2026	Confirmed	<code>getTrancheCount(C2)</code> =10; tranche 0 TIME_ONLY, <code>unlockTime</code> =1785283200
"All contracts governed by 48h timelock + Safe"	Mostly confirmed, with named exceptions	every contract owner/admin = Governance (incl. BuybackController — docs stale in the safe direction); delay 172,800 s; 3× Safe 3-of-5 with identical 5 owners. Exceptions: all guardian roles = single EOA; <code>setGuardian</code> untimelocked; voucher signer = hot role
Bootstrap finalized, 3 contributors, 0.03 ETH, LP retained	Confirmed	<code>getBootstrapStatus()</code> = (false, true, 0.03e18, 0, 3); LP custody verified (W17)
"Unlock always available, even when paused" (IFRLock)	Confirmed (source)	<code>unlock()</code> has no <code>whenNotPaused</code> ; <code>_lock</code> does
Vesting cliff ~03.2027, 150M, beneficiary immutable	Confirmed — wiki start value is a typo	chain: <code>beneficiary</code> = 0x04FA...6239, <code>start</code> = 1772670647 (2026-03-05T00:27 UTC), <code>released</code> = 0
Slither 0 unreviewed High / 0 Critical; Mythril clean	CI-attested	<code>security-audit.yml</code> green at HEAD push (13:15Z); <code>mythril-analysis.yml</code> green (12.09)
No secrets in repo	Confirmed (sweep + CI)	no tracked <code>.env</code> ; zero 32-byte-hex hits outside docs/patches; dummy CI secrets only; gitleaks gate green
npm audit clean	Confirmed at moderate+	8 low (transitive @ethersproject/* via hardhat-verify), 0 moderate+
Fee tiers 200/50/100 bps, cap 500	Confirmed on-chain	token reads @ 25971217
Fee-exempt list (10 entries, docs)	Diverged	chain: 15 exempt of the audited 18 (incl. Vesting, BootstrapVaultV3, both big Safes, Deployer); LP Reserve Safe, Governance, BuilderRegistry not exempt
PartnerVault rewardBps 1000 / throttle active	Diverged	chain: <code>rewardBps</code> = 1500, <code>ifrLock</code> = 0x0 (throttle disabled); annual cap 4M ✓
Lending "audited smart contract" wording	Overstatement	only internal/AI/community reviews exist; this report is the first external one

7. Governance & Centralization Risk

Verified roles matrix: see §3. Scenario analysis:

- **Single malicious proposal:** any protocol parameter change is visible on-chain 48h ahead; guardian cancel is the reactive backstop — held by one EOA (CWA-09). The highest-impact single proposal is `setIFRPrice` (CWA-01). With lending disabled, no proposal can directly move user funds; the largest movable balances are LiquidityReserve withdrawals (50M/period cap, CWA-15) and reserve/buyback parameters.
- **Compromised guardian EOA:** can cancel all proposals and pause every pausable contract — sustained griefing, no theft. Recovery requires Safe actions (unpause via new guardian rotation, which is instant).
- **Compromised voucher-signer hot key:** forged discount vouchers (bounded: 15 bps off a 5 bps fee — economically near-zero) — but if that key is also the Safe owner key (CWA-06), the blast radius jumps to "1 of 3 required Safe keys".
- **Signer set:** 5 pseudonymous signers, identical across all 3 Safes (verified). Documented expansion to 4-of-7 is not yet on-chain. One signer's EOA doubles as the public infrastructure donation address (disclosed) — noted as an opsec correlation point, not a vuln.
- **Proposal audit trail:** `proposalCount()` = 17 matches the documented #0–#16 log; spot-decoded #15 (`setFeeExempt(LP pair, true)`) and #16 (`setP0(300,000,000)`) match the wiki's governance log exactly — the on-chain record and the public log agree.

8. Web & API Security

- **Header matrix (re-verified 2026-09-13):** shop = strong (enforced CSP + report-only, XFO DENY, HSTS 2y incl. subdomains, nosniff, no-referrer, restrictive Permissions-Policy); apex Pages = platform-limited (no custom headers possible; ACA0: * is Pages default on a static site — harmless); web3 = none (CWA-10); verify-api = ACA0: * + default-src 'none' (acceptable for a POST-only service; preflight-only exposure); points-api = default-src 'none' + nosniff; copilot-api = vary: Origin allowlist (verified in source).
- **Address integrity:** every address embedded in homepage (21 unique), `/web3/` (apex and subdomain — byte-identical, confirmed via `cmp`), and `builder.html` matches the verified deployment map. No rogue or swapped addresses. `c2Addr` hard-coding is consistent with the transparency page.
- **XSS review:** copilot widget renders all dynamic chat content via `textContent` (safe); web3 `innerHTML` usages interpolate only numeric/hex contract data and static page constants — no attacker-controlled strings reach a sink on the reviewed paths. No external `<script>` tags without SRI on the homepage (only CSS/fonts).
- **API behavior:** all reviewed endpoints validate addresses (`/^0x[0-9a-fA-F]{40}$/`), bound pagination, and fail closed (502 with generic error; the price endpoint's live 502 confirms the no-stale-data behavior). API-reported supply/balances matched chain reads to the unit at baseline (single-source RPC caveat: the API reads `latest`, we pinned a block — figures agreed). Copilot health endpoint disclosure: CWA-11. points/verify source findings: CWA-04/05/14.
- **Not done:** authenticated flows, POST endpoints, and robots-disallowed shop paths (`/api/ /b/ /p/ /r/`) were not probed (read-only policy); shop backend business logic reviewed at middleware level only.

9. Repository & Supply Chain

- Tests: full suite reproduced (`npm ci clean install` → 644 + 30 + 36 passing, exit 0) at the pinned commit — the strongest claims-verification result of this audit.
- CI: 16 workflows, **all actions SHA-pinned**, `permissions: contents: read` (one adds `pull-requests: read`), Node 22; green runs at/around HEAD for contracts, security-audit, Mythril, docs, and the 4-hour vault-invariant monitor (last success 18:49Z, matching the claimed cadence). Enforcement gap: CWA-07.
- Secrets: no `.env` tracked (12 `.env.example` files, placeholders only); pattern sweep for private-key/mnemonic/API-key shapes found zero real secrets outside docs/patches; CI-only dummy secrets are clearly labeled; `.gitleaks.toml` has sensible custom rules; CI gitleaks gate green at HEAD. Note `.env.example`'s `DEPLOYER_PRIVATE_KEY=0xYOUR_PRIVATE_KEY_HERE + "TODO: REMOVE after governance transfers"` — transfers appear complete (owner=Governance everywhere), so the TODO's precondition looks met: remove the placeholder key name entirely.
- Dependencies: `npm audit --audit-level=moderate` → 0 findings (8 low, transitive via hardhat-verify). Dependabot active; 3 version-bump PRs open.
- Branch protection: 1 approval + linear history + no force-push, but no required status checks and `enforce_admins: false` (CWA-07); no required commit signatures (optional hardening).

10. Recommendations (prioritized)

1. **Keep lending fail-closed** until CWA-01 remediations ship (bounded/TWAP price, borrow caps, `protocolFeeReceiver`, overdue-loan path, `.call` collateral returns). Publish the activation runbook as a governance prerequisite.
2. **Resolve the FeeRouterV1 sink (CWA-02)**: decide sink-vs-sweep, then either re-point `poolFeeReceiver` via timelock or correct the three doc locations that promise a flywheel.
3. **Guard CommitmentVault price tranches (CWA-03)**: revert on non-TIME_ONLY until a real oracle exists; fix the wiki wording.
4. **Fix the points-backend auth trio (CWA-04/05/06)**: SIWE domain binding, production-throw on Sepolia defaults, dedicated voucher-signer key.
5. **Enforce the CI gates on main (CWA-07)** and enable `enforce_admins`.
6. **Complete the guardian-multisig migration (CWA-09)** — the single remaining broad EOA power.
7. **Documentation pass (CWA-20/21)**: regenerate every number from chain state at publish time (the project already has `update-stats.yml` — extend it to tier tables, test counts, fee-exempt table, and the copilot welcome strings), fix the vesting `start` typo, align the license.
8. Low-effort hardening: web3 headers (CWA-10), `/api/health` minimization (CWA-11), zero-address requires (CWA-18), two-step ownership in future versions (CWA-16), header/secret hygiene for `.env.example` TODO (§9).

11. Conclusion

The project's self-assessment — "internal audits done, professional audit pending, lending deliberately fail-closed" — is **confirmed as accurate** by this first external review. The code quality, test reproducibility, CI discipline, and the honesty of the known-issues register are above the norm for a project of this size, and every load-bearing safety claim (timelock, multisig, fail-closed states, LP lock, test counts) was verified true on-chain or in execution.

The gaps are concentrated where the project itself signals they would be: the dormant lending price mechanism (do not activate as-is), one real accounting dead-end (FeeRouterV1 IFR sink), young off-chain components (points/voucher auth hardening), and a documentation layer whose numbers drift in both directions — overstated in places (tier tables, "audited" wording, stale supply/fee tables) and *understated* in others (BuybackController ownership migration, Vesting fee exemption, W18 fix). With one qualification, none of the findings puts user funds at risk in the current configuration: locks newly created through the externally callable `CommitmentVault.lock()` with `PRICE_ONLY` or `TIME_AND_PRICE` may be unrecoverable while the oracle remains zero, because `isConditionMet()` cannot succeed and neither `unlock()` nor the permissionless auto-unlock can release them (CWA-03). The existing `TIME_ONLY` baseline tranches (47,952,476.87 IFR) are unaffected.

12. Disclaimer

This report is a point-in-time security review limited to the scope and baselines stated above (repo commit `eb538a35...`, mainnet block 25971217, surfaces as of 2026-09-13). It is based on read-only analysis of public source code, public on-chain state, and public web surfaces. It does not guarantee the absence of vulnerabilities, does not cover private infrastructure or operational security, and does not constitute financial advice, an endorsement, or a legal compliance opinion (notably not on §5 DDG/DSGVO/MiCA questions raised by the community audit). On-chain state changes with every block; readers must re-verify roles and balances before relying on them.

Appendix A — Address & endpoint inventory

Verified against chain state at block 25971217 unless noted:

Address	Component	Owner/Admin	Verified state highlights
0x77e99917Eca8539c62F509ED1193ac36580A6e7B	InfernoToken	Governance	fees 200/50/100 bps; poolFeeReceiver = FeeRouterV1; supply 997,571,140.022456196
0xc43d48E7FDA576C5022d0670B652A622E8caD041	Governance	Treasury Safe	delay 172,800 s; guardian = Deployer EOA; proposalCount 17
0x769928aBDfc949D0718d8766a1C2d7dBb63954Eb	IFRLock	— (guardian Deployer)	totalLocked 2,000 IFR; not paused
0x2694Bc84e8D5251E9E4Ecd4B2Ae3f866d6106271	Vesting	immutable beneficiary	150M; start 1772670647; released 0; feeExempt = true
0xdc0309804803b3A105154f6073061E3185018f64	LiquidityReserve	Governance	200M; lockEnd 1788222659 (passed); 50M/90d; withdrawn 0
0x670D293e3D65f96171c10DdC8d88B96b0570F812	BuybackVault	Governance	0 ETH / 0 IFR
0xaA1496133B6c274190A2113410B501C5802b6fCF	BurnReserve	Governance + guardian	totalBurned 0
0xc6eb7714bCb035ebc2D4d9ba7B3762ef7B9d4F7D	PartnerVault	Governance	40M; rewardBps 1500; cap 4M; ifrLock unset

Address	Component	Owner/Admin	Verified state highlights
0x4807B77B2E25cD055DA42B09BA4d0aF9e580C60a	FeeRouterV1	Governance (immutable)	fee 5 bps; feeCollector = BuybackController; 371,543.99 IFR stranded
0xf72565C4cDB9575c9D3aEE6B9AE3fDBd7F56e141	BootstrapVaultV3	none	finalised; 0.03 ETH; 3 contributors; holds 54,772,255.75 LP (all minus 1000 wei)
0xdfe6636DA47F8949330697e1dC5391267CEf0EE3	BuilderRegistry	Governance	0 builders
0x0719d9eb28dF7f5e63F91fAc4Bbb2d579C4F73d3	CommitmentVault	Governance	47,952,476.87 IFR; p0 = 3e8; priceOracle = 0
0x974305Ab0EC905172e697271C3d7d385194EB9DF	LendingVault	Governance	52,155,440.95 IFR over 3 offers; 0 loans; ifrPriceWei = 0; protocolFeeReceiver = 0
0x1e0547D50005A4Af66AbD5e6915ebfAA2d711F7c	BuybackController	Governance (docs stale)	0 ETH; cooldown 86,400 s; slippage 500
0xbE495E9c0d8cc2DCf95570cf95B63c4844dF31A0	Uniswap V2 pair	—	feeExempt = true (Proposal #15, decoded on-chain)
0x5ad6193eD6E1e31ed10977E73e3B609AcBFecE3b	Treasury Safe	3-of-5 (owners listed §3)	= Governance owner
0xaC5687547B2B21d80F8fd345B51e608d476667C7	Community Safe	3-of-5 (same set)	7.9M IFR
0x5D93E7919a71d725054e31017eCA86B026F86C04	LP Reserve Safe	3-of-5 (same set)	400.6M IFR; not feeExempt

Endpoints: copilot-api (/api/health, /api/ifr/{supply,balances,check,price,txfeed,vault}, /api/locks/events, /api/lending/*, /api/commitment/*, /api/builder/generate, /api/bootstrap/vote[s], /api/builders/*); points-api (/health live; SIWE/points/voucher per source); verify-api (404 on /, preflight only); shop (/api/health 200 chainId 1).

Appendix B — Tooling

cast/foundry 1.7.1 (drpc + publicnode, block-pinned reads) · node v22.22.2 / npm 10.9.7 (clean npm ci + npm run test:contracts:all, exit 0) · gh API (repo metadata, protection, CI run history) · curl (header/endpoint matrix) · CI attestations: contracts.yml / security-audit.yml success @ HEAD push 2026-09-13T13:15:20Z; mythril-analysis.yml success 2026-09-12T22:21:33Z; vault-invariant-monitor.yml success 2026-09-13T18:49:10Z. Not locally executed: Slither 0.11.5, Mythril 0.24.8, gitleaks (CI-attested instead — see §2 Limitations).

Appendix C — Verification commands

All chain reads were executed as:

```
cast call --block 25971217 --rpc-url https://eth.drpc.org <address> "<signature>" [args]
```

Full labeled outputs: `evidence/chain-reads-correct-block.txt`, `evidence/chain-reads-extended.txt`, `evidence/final-onchain-checks.txt`, `evidence/governance-c2-checks.txt`. Test reproduction: `evidence/test-run.log`. Web/API observations: `evidence/web-api-recheck.txt`, `evidence/web-address-integrity.txt`. Repo/CI: `evidence/repo-ci.txt`, `evidence/repo-secrets-deps.txt`. Baselines: `evidence/baseline-*.txt`.

— Collateral Web3 Open Audits, 2026-09-14

IFR Protocol — Contract Deep Security Audit

Auditor: Collateral Web3 Open Audits **Client:** NeaBouli / Inferno Protocol (ifrun.it.tech) **Date:** 2026-09-14
Baselines: repo NeaBouli/inferno @ eb538a355001b042b343bfb19221af2407a96e63 · Ethereum mainnet block **25971217** (pinned 2026-09-13T21:29:57Z) · Foundry 1.7.1 · Slither 0.11.5 · Mythril 0.24.8 · solc 0.8.20 (a1b79de6) / 0.8.28 **Companion documents:** CWA_IFR_Protocol_Audit_2026-09-14.md (full-scope audit, CWA-01...24), CWA_IFR_Surfaces_Supplement_2026-09-14.md (remaining app surfaces, CWA-28+) **Report version:** 1.0

This document is the contract-focused deep audit: proof-grade verification of the 14 deployed mainnet contracts. Where the full-scope audit reviewed behavior, this audit proves it — with reproduced tool runs, fuzzed invariants, bytecode comparison against deployed code, and block-pinned chain reads.

1. Executive summary

All six evidence layers were executed end-to-end. The contract suite is in strong shape: Slither and Mythril gates were **reproduced locally** (not just CI-attested), a purpose-built Foundry harness fuzzed the load-bearing invariants (14/14 passing, 256 runs per property), and the deployed bytecode was compared against locally compiled source for every contract.

The most important result is not a new bug but a **precision correction**: two documented "fixed" issues are fixed **in the repository source only — the deployed mainnet bytecode predates both fixes**. Governance's `setOwner` is still directly callable by the owner on-chain (CWA-25, four independent proofs), and BuybackVault's `setParams` bounds are absent on-chain (CWA-26, era-source diff). Neither creates an immediate fund risk (both sit behind the TreasurySafe 3-of-5 and, for parameter changes, the 48h timelock), but the public security record currently says "fixed" where the chain says "not deployed".

No new vulnerability was found in the deployed contract logic itself.

Result: 0 Critical · 0 High · 1 Medium · 1 Low · 2 Informational (this document's new findings; register continues from the full-scope audit)

2. Scope, baselines, tooling

In scope: the 14 deployed mainnet contracts (addresses in Appendix A of the full-scope report), their sources at eb538a35, their deployed bytecode at block 25971217, and the protocol's safety invariants.

Tools and evidence files (relative to the audit workspace; the PR contains the fuzz harness):

- Full test suite reproduction: `npm ci → npm run test:contracts:all → 644 + 30 + 36 passing, exit 0` (evidence/test-run.log).
- Slither 0.11.5 (venv, solc 0.8.28) via the project's own gate: `npm run test:slither-baseline PASS + npm run check:slither → "analyzed 21 production Solidity files · observed 6 reviewed High signals · PASS — no new or changed High signals", exit 0` (evidence/slither-run.log).
- Mythril 0.24.8 (Python 3.12 venv, solc 0.8.28) via the project's own gate: `npm run check:mythril → 0 signals on all 17 concrete contracts`, branch coverage 77.8–97.8% per contract, "PASS — no Critical, High or Medium signals in the bounded scope", exit 0 (evidence/mythril-run.log; bounds: 2 tx / 30 s

execution / 10 s solver per `audit/mythril-config.json`).

- Foundry 1.7.1 fuzz harness (`evidence/IFRDeepAudit.t.sol`, pinned-commit sources imported read-only; solc 0.8.28/cancun for the harness because current OZ `EIP712.sol` requires $\geq 0.8.24$ — logic-equivalent for the tested properties): **14/14 passing** (`evidence/fuzz-run.txt`).
- Bytecode comparison: solc 0.8.20/paris/no-opt local builds vs. `cast` code at block 25971217, CBOR-metadata-stripped (`evidence/bytecode-compare.txt`), plus era-source diffs from Blockscout's verified-source packages and git history per file.
- Explorer verification sweep: **all 14 contracts verified on Blockscout** (name, compiler v0.8.20+commit.a1b79de6, no optimization, paris — per-contract records in `evidence/blockscout-verification-sweep.txt`); InfernoToken additionally confirmed on Etherscan.

Could not be run locally: gitleaks binary (CI gate green at HEAD + local pattern sweep instead); Etherscan's API without a key (Blockscout used as the independent second source).

3. Per-contract results

Contract	Manual review	Slither	Mythril	Fuzz/chain	Result
InfernoToken	clean	baseline	0 signals	fee conservation + exemption fuzz PASS	CLEAN
Governance	1 finding	baseline	0 signals	live probe proves CWA-25	MEDIUM (CWA-25)
IFRLock	clean	baseline	0 signals	accounting fuzz PASS; drift proof PASS; deployed source byte-identical to HEAD	CLEAN
Vesting	clean	baseline	0 signals	monotonicity/cap fuzz PASS; <code>transferGuardian</code> exists on-chain	CLEAN
LiquidityReserve	clean (cap mutability noted in CWA-15)	baseline	0 signals	period-cap fuzz PASS	CLEAN
BuybackVault	1 finding	baseline	0 signals	era-source diff proves CWA-26	LOW (CWA-26)
BurnReserve	clean	baseline	0 signals	chain: burn-only surface	CLEAN
PartnerVault	clean	baseline	0 signals	chain: bounds active (rewardBps 1500 $\in$ [500,2500], cap 4M)	CLEAN
FeeRouterV1	known W10/W11/W19 confirmed in source	baseline	0 signals	voucher lifecycle fuzz PASS (7 cases)	CLEAN w/ known Lows (CWA-18)
BootstrapVaultV3	clean	baseline	0 signals	finalized on-chain; LP custody proven	CLEAN (W17 verified)
BuilderRegistry	clean	baseline	0 signals	chain: 0 builders	CLEAN
CommitmentVault	CWA-03 confirmed + fuzz-proofed	baseline	0 signals	price-tranche permanence proven	known MEDIUM (CWA-03)
LendingVault	CWA-01/08 confirmed in source	baseline	0 signals	chain: fail-closed	known HIGH-dormant (CWA-01)

Contract	Manual review	Slither	Mythril	Fuzz/chain	Result
BuybackController	clean	baseline	0 signals	chain: owner=Governance, 0 ETH	CLEAN

4. Findings

Medium

[CWA-25] Deployed Governance predates the W1 fix: `setOwner` bypasses the timelock on mainnet

- **Severity:** Medium · **Likelihood:** low (requires the 3-of-5 Safe itself) · **Status:** Open on-chain / fixed in source
- **Component:** `Governance.sol` (`0xc43d48E7FDA576C5022d0670B652A622E8caD041`)
- **Cross-refs:** W1 (documented as fixed); full-scope audit CWA-09 (guardian model)
- **Description:** The repo source (since commit `67065174` , 2026-03-20) routes `setOwner` through the timelock (`onlySelf`). The deployed contract (deployed 2026-03-05) still carries the original `onlyOwner` version, so the TreasurySafe can replace Governance ownership **immediately, without a 48h proposal window and without the guardian's cancel option**.
- **Proofs (four independent):**
 1. Live read at block 25971217: a `setOwner(address)` call from an arbitrary address reverts `"not owner"` — the `onlyOwner` gate text, not the `onlySelf` gate text (`"not self"`).
 2. Bytecode comparison: deployed 8,016 B vs. HEAD-compiled 7,984 B (no constructor immutables exist in this contract, so the difference is real code drift).
 3. Commit `67065174` diff: `- function setOwner(address _owner) external onlyOwner → + ... onlySelf` — dated 15 days *after* the 2026-03-05 deployment.
 4. Blockscout era-source package for the deployed address contains the `onlyOwner` variant.
- **Impact:** the honest residual risk is narrow. The Safe already holds absolute power through the timelock; what is lost for this one transition is the **48h public window and the guardian-cancel backstop** on the single most sensitive action (ownership transfer). All subsequent parameter changes by a new owner would still be timelocked (`propose` / `execute` unchanged). No direct fund movement is enabled beyond what the Safe can already do.
- **Recommendation:** publish a one-line correction wherever W1 is listed as "fixed" ("fixed in source; live Governance carries the pre-fix variant; ownership is with the 3-of-5 Safe; a Governance redeploy would change its address and require migrating every child contract's owner pointer"). If a Governance V2 is ever deployed, migrate deliberately with a public checklist. Do not hot-swap.

Low

[CWA-26] Deployed BuybackVault predates the W3 bounds fix (`setParams` unbounded on-chain)

- **Severity:** Low · **Likelihood:** low (governance-error path only; contract holds 0 ETH) · **Status:** Open on-chain / fixed in source
- **Component:** `BuybackVault.sol` (`0x670D293e3D65f96171c10DdC8d88B96b0570F812`)
- **Cross-refs:** W3
- **Description:** era-source diff (Blockscout verified package vs. HEAD) shows exactly three added lines in HEAD: `require(_burnShareBps <= 10_000)` , `require(_slippageBps <= 10_000)` , `require(_router != address(0))` — committed 2026-03-10 ("fix: BuybackVault — add bounds validation to `setParams()`"),

five days after deployment. On-chain, a timelocked governance proposal could set `slippageBps > 10000` or `burnShareBps > 10000`; both paths then revert on underflow during `executeBuyback` — a buyback DoS until re-parameterized, not a fund-loss path (and the contract holds 0 ETH / 0 IFR at baseline).

- **Recommendation:** same class of action as CWA-25: correct the register wording ("fixed in source, not deployed"), fold into any future V2. No urgency while unfunded.

Informational

[CWA-27] Bytecode verification classification (methodology result, not a defect)

- **Byte-exact match (HEAD ↔ deployed):** InfernoToken, BuilderRegistry.
- **Deployed source byte-identical to HEAD** (per Blockscout era packages + git history: no post-deploy edits); residual bytecode differences fully explained by constructor-baked immutables and/or OpenZeppelin dependency drift between the deployment-era lockfile and the current one: IFRLock (proven byte-identical source), Vesting, LiquidityReserve, BurnReserve, BuybackController, PartnerVault, BootstrapVaultV3, CommitmentVault, LendingVault.
- **Real drift:** Governance (CWA-25), BuybackVault (CWA-26).
- **Locally inconclusive:** FeeRouterV1 — current `@openzeppelin/contracts@5.6.1 EIP712.sol` requires `solc ≥ 0.8.24`, so a 0.8.20 local rebuild cannot reproduce the deployment build; the contract is dual-explorer verified (Blockscout + Etherscan) with the era's dependency set. Recommend adding the deployment-era `package-lock.json` reference to `docs/DEPLOYMENTS.md` so future bytecode reproductions stay possible.

(The fuzz harness itself is delivered alongside this report as a permanent evidence artifact; it carries no finding ID — the supplement document continues the register from CWA-28.)

5. Invariant table

#	Property	Method	Result
I1	<code>net + poolFee + burns == value</code> on every non-exempt transfer; <code>totalSupply</code> only decreases	fuzz (256 runs, random fee rates within cap + amounts)	HOLDS
I2	<code>feeExempt</code> transfer moves the full amount and burns nothing	fuzz (256 runs)	HOLDS (documents exemption blast radius)
I3	IFRLock: <code>balanceOf == totalLocked == Σ user locks</code> under random lock/unlock sequences	fuzz (256 runs, 1–24 actions, 3 users)	HOLDS — with exemption set
I4	IFRLock without <code>feeExempt</code> : user is credited more than the contract receives and cannot unlock	targeted proof test	PROVEN — the exemption is load-bearing; governance must never remove it while locks exist
I5	Vesting: <code>vestedAmount</code> non-decreasing, $\leq$ allocation, 0 before cliff, full after duration; <code>release()</code> pays exactly vested	fuzz (256 runs over time pairs)	HOLDS
I6	LiquidityReserve: pre-lock withdrawals revert; per-period cap holds to the wei; 1 wei over cap reverts	fuzz (256 runs, 1–4 periods)	HOLDS

#	Property	Method	Result
17	CommitmentVault: PRICE_ONLY tranche can never unlock — not by owner, not by anyone, even with oracle set, even after 2,000 days	targeted proof test	PROVEN (CWA-03 formalized); TIME_ONLY control unlocks correctly
18	FeeRouterV1 voucher: valid discount executes; replay reverts; wrong signer reverts; discount > fee reverts; expiry reverts; no-voucher path charges exactly 5 bps	6 targeted tests	HOLDS
19	FeeRouterV1 with <code>feeCollector = 0x0</code> : fee ETH is silently burned (call succeeds)	targeted proof test	PROVEN (CWA-18 mechanism confirmed)

6. Tool-run evidence

- Slither: [slither] analyzed 21 production Solidity files · observed 6 reviewed High signals · PASS — no new or changed High signals (exit 0) — locally reproduced with the project's pinned venv recipe; baseline file matches the CI claim.
- Mythril: per-contract table in `evidence/mythril-run.log`; all 17 concrete contracts **0 signals** under the project's bounded config; worst branch coverage 77.8% (IFRBuilderVault library module), core contracts 85–98%.
- Repo test suite: 644 contract + 30 builder + 36 SDK tests passing at `eb538a35`.
- Fuzz: 14/14 PASS (`evidence/fuzz-run.txt`); harness `evidence/IFRDeepAudit.t.sol` (reproducible with the remappings in `evidence/fuzz-foundry.toml`).

7. Known-issues register — precision corrections

Item	Previously documented	Deep-audit status
W1 (<code>setOwner</code> timelock bypass)	"fixed"	Fixed in source only; NOT deployed on mainnet (CWA-25)
W3 (unbounded <code>setParams</code>)	"partially addressed"	Fixed in source only; NOT deployed on mainnet (CWA-26)
W18 (Vesting guardian not rotatable)	open	Outdated — <code>transferGuardian</code> present in source and in deployed bytecode (selector probe reverts <code>OnlyGuardian()</code> for strangers)
W6 (Vesting pays 3.5% fees)	accepted	Superseded — <code>feeExempt[Vesting] = true</code> on-chain
W17 (LP stranded in <code>BootstrapVaultV3</code>)	accepted	Verified — <code>lpLockId = 0</code> , vault holds all LP minus the 1000-wei Uniswap minimum; no withdrawal function exists (by design)
OPS-001 / OPS-002	fail-closed	Verified on-chain (<code>ifrPriceWei = 0</code> , <code>priceOracle = 0x0</code>)

8. Recommendations (prioritized)

- Correct the public register wording for W1/W3 ("fixed in source; deployed variant differs; no immediate risk because ..."). Precision here is the project's credibility engine.
- Add the deployment-era dependency lockfile reference to `docs/DEPLOYMENTS.md` so bytecode reproduction stays possible (CWA-27).

3. Before any funding of BuybackVault/BuybackController, redeploy or accept the on-chain absence of `setParams` bounds (CWA-26) explicitly in the funding proposal text.
4. Add the fuzz harness (delivered with this report) to the repo's test tree or CI as a permanent invariant gate — especially I4 (feeExempt dependency), which is the protocol's single load-bearing operational assumption.
5. If a Governance V2 ever ships: migrate via public checklist, never hot-swap; every child contract's owner pointer must move in one governed sequence.

9. Limitations & disclaimer

Point-in-time review at the stated baselines. Symbolic execution is bounded (2 tx / 30 s / 10 s solver) — "0 signals" means "no signal within the bounded scope", not proof of absence. Fuzzing covers the nine invariants in §5, not all possible properties. The bytecode method proves equivalence classes (exact match / immutable-slot drift / dependency drift / real drift); the two real-drift cases were additionally proven by live probes and era-source diffs. This report is not a certification, not financial advice, and no guarantee that no vulnerability exists. On-chain state changes with every block — re-verify before relying.

— Collateral Web3 Open Audits, 2026-09-14

IFR Protocol — Surfaces Supplement: creator-gateway, Telegram bot, Benefits backend, SDK, dashboards

Auditor: Collateral Web3 Open Audits **Client:** NeaBouli / Inferno Protocol (ifrunit.tech) **Date:** 2026-09-14
Baseline: repo NeaBouli/inferno @ eb538a355001b042b343bfb19221af2407a96e63 **Companion documents:** CWA_IFR_Protocol_Audit_2026-09-14.md (CWA-01...24), CWA_IFR_Contracts_Deep_Audit_2026-09-14.md (CWA-25...27) **Method note:** this supplement is a **static source review** of the listed applications at the pinned commit (AI-assisted analysis inside the CWA audit pipeline, with an independent review pass over every security-relevant file — see §5 for the reviewed/skipped list). No dynamic testing, no requests against live deployments beyond public GETs; deployment status of creator-gateway was not observable from public surfaces and must be checked before relying on the exploitability statements.

This document completes the full-scope audit: the first report explicitly left the app's business logic beyond the three API backends unreviewed. This supplement closes that gap.

1. Executive summary

The remaining surfaces split into three quality tiers. The **Benefits backend** is in good shape — redemption atomicity, IDOR protection, challenge single-use, and fail-closed rate limiting are all implemented correctly; findings there are Low. The **Telegram bot** has one coherent Medium cluster around wallet verification (entropy, bearer-code exposure, missing wallet uniqueness, deanonymizing status endpoint) plus Low hygiene items. **creator-gateway** contains the two most serious findings of this supplement: both the legacy `/auth/wallet` endpoint and the Google OAuth flow issue JWTs whose `walletAddress` claim is fully attacker-chosen — a complete bypass of the "lock $\geq$ 1,000 IFR" entitlement gate, as-coded.

New findings: 2 High · 5 Medium · 8 Low · 3 Informational (CWA-28 ... CWA-45)

2. Findings

High

[CWA-28] creator-gateway: unauthenticated JWT minting via legacy `/auth/wallet` — full entitlement bypass

- **Severity:** High · **Likelihood:** high if the service is publicly deployed (not confirmed) · **Status:** Open — works as-coded
- **Component:** `apps/creator-gateway/src/routes/auth.ts:112-133`, consumed by `routes/access.ts:8-21` → `services/entitlement.ts:33-36`
- **Description:** `POST /auth/wallet` accepts any well-formed address — no signature, no nonce, only `ethers.getAddress` format validation — and returns a 24h JWT with that `walletAddress`. `/access/check` then evaluates the on-chain lock status **of the claimed wallet**. Any wallet with $\geq$ 1,000

IFR locked (public data) yields `granted: true` for the attacker. The endpoint is live by default and merely marked deprecated via an `X-Deprecated` response header.

- **Impact:** complete bypass of the protocol's central product mechanic (lock-to-access) for everything behind this gateway. Not a fund-theft path — hence High, not Critical.
- **Recommendation:** disable `/auth/wallet` outright (remove the route, not just the header). If a legacy transition window is truly needed, gate it behind an env flag that defaults off.

[CWA-29] creator-gateway: Google OAuth flow binds an arbitrary wallet into the JWT

- **Severity:** High · **Likelihood:** high if deployed · **Status:** Open — works as-coded
- **Component:** `apps/creator-gateway/src/routes/auth.ts:27-64`
- **Description:** `/auth/google?wallet=<address>` carries the wallet in the OAuth `state` parameter with no proof of ownership; the callback validates only the address format and issues a JWT containing `walletAddress: <victim>` alongside the attacker's Google tokens. Equivalent impact to CWA-28 through the "supported" path. Additionally there is no OAuth `state` CSRF nonce (login-CSRF class).
- **Recommendation:** bind the wallet only after a SIWE-verified session exists: OAuth establishes the Google identity, SIWE proves the wallet, and the JWT is minted only for the pair. Add a random `state` nonce stored server-side.

Medium

[CWA-30] creator-gateway: SIWE verification omits domain binding (third instance of this class)

- **Severity:** Medium · **Likelihood:** medium (needs victim signature on a phishing site) · **Status:** Open
- **Component:** `apps/creator-gateway/src/routes/auth.ts:84` — `siweMessage.verify({ signature })` without `{ domain }`; nonce store is correct (server-side, single-use, `:87-91`), `chainId` is checked (`:94`).
- **Description:** same bug class as CWA-04 (points-backend) but a different codebase and deployment: a SIWE message signed for any other domain is accepted, so a harvested phishing signature becomes a JWT for the victim's wallet.
- **Recommendation:** pass `{ domain: <expected> }` to `verify()`. Fix all three instances (CWA-04, CWA-30, and check any future SIWE consumer) in one sweep — this class propagates by copy-paste.

[CWA-31] Telegram verify: nonce from `Math.random` (~31 bits) and no rate limit on `/api/verify`

- **Severity:** Medium · **Likelihood:** low-medium · **Status:** Open — works as-coded
- **Component:** `apps/telegram/telegram-bot/src/services/verificationStore.js:117-119`, `index.js:212-269`
- **Description:** the verify nonce is `'IFR-' + Math.random().toString(36).substr(2,6) + timestamp` (~ 2^{31} random space, predictable `Math.random` state, predictable timestamp), TTL 10 minutes, and the verify API has **no per-IP/per-nonce attempt limit**. A guessed nonce lets an attacker bind their wallet to someone else's Telegram identity. The cryptographic check itself is correct (EIP-191 `verifyMessage` against the claimed wallet, single-use, TTL'd).
- **Recommendation:** CSPRNG nonce (`crypto.randomBytes`, ≥ 128 bits), per-IP + per-nonce attempt counters, exponential backoff on failures.

[CWA-32] Telegram verify: code is a first-come bearer token, often posted publicly; victim lockout

- **Severity:** Medium · **Likelihood:** medium · **Status:** Open — works as-coded
- **Component:** `apps/telegram/telegram-bot/src/commands/verify.js:28-44` (`ctx.reply` in whatever chat invoked `/verify`), `index.js:235` (first submission wins), `verify.js:17-26` ("Already verified" short-circuit, no unbind path)
- **Description:** a `/verify` invoked in the public group exposes the code to everyone; whoever submits it first binds *their* wallet to the victim's Telegram ID, and the victim can never complete verification afterwards (no unbind/re-verify flow).
- **Recommendation:** answer `/verify` only in DM (delete the group invocation), treat the code as a secret, add an admin-gated unbind path.

[CWA-33] Telegram verify: one wallet can verify unlimited Telegram accounts

- **Severity:** Medium · **Likelihood:** medium · **Status:** Open — works as-coded
- **Component:** `services/verificationStore.js:156-167` — maps keyed by `userId` only; no uniqueness on the wallet side.
- **Description:** a single qualifying wallet (e.g. a signer wallet) can mint unlimited "verified" Telegram identities into the Core Dev (58), Council (21), Vote (23) and Dev & Builder (11) topics — defeating the tier gate as a Sybil control.
- **Recommendation:** enforce `wallet → at most one active userId` (unique index + re-verify moves the binding).

[CWA-34] Telegram verify: unauthenticated `/api/verify/status/:userId` leaks wallet↔Telegram linkage

- **Severity:** Medium · **Likelihood:** certain (public) · **Status:** Open — works as-coded
- **Component:** `apps/telegram/telegram-bot/src/index.js:271-275`
- **Description:** anyone can enumerate Telegram user IDs and read `{verified, wallet, tier}` with full (unmasked) wallet addresses — a deanonymization oracle for every verified member. The project's own wiki promises privacy-preserving verification; this endpoint contradicts it.
- **Recommendation:** drop the wallet from the public response (or return a salted hash), add rate limiting, consider requiring the request to prove control of the Telegram ID (bot-signed token).

Low

- **[CWA-35] Benefits backend:** read-only seller actions (`business:list`, `sessions:list`, `rules:list`, `products:list`, `operators:*`, `rewards:read`) skip the single-use challenge — a captured signature triple replays for ≤ 10 minutes (`services/sellerAuthorizationActions.ts:2-9`). Mutations are properly challenge-gated; residual read-replay window only.
- **[CWA-36] Benefits backend:** seller authorization message lacks `Domain: /Chain ID: binding` (unlike the customer challenge, which has both) — same signature is valid on any deployment sharing the format (`services/sellerAuth.ts:19-36` vs. `sessionService.ts:445-468`).
- **[CWA-37] Benefits backend:** attest grieving — anyone holding a session ID can burn the 3 signature attempts (`sessionService.ts:635-694`; the ID also transits unmasked via public `GET /api/sessions/:id`); 50/h/IP limiter slows but doesn't stop it. Bounded: kills one in-flight checkout; seller can recreate.

- **[CWA-38] Telegram:** hardcoded privilege fallbacks activate when env is unset — `PROTECTED_ADMIN_IDS` defaults to a literal user ID `579949616` (`index.js:130–132`); on Safe RPC failure **and** unset `SIGNER_WALLETS`, signer tier falls open to the hardcoded deployer EOA (`services/onChainReader.js:48–55`). Fail-closed on missing config would be safer.
- **[CWA-39] Telegram verify CORS:** substring origin match (`origin.includes('ifrunit.tech')`) accepts `ifrunit.tech.evil.example`; empty Origin gets `*`). Contrast: `communityLinks.js` does proper origin parsing — apply the same rigor (`index.js:202–209`).
- **[CWA-40] Telegram:** the protected-topic guard (`bot.on('message')`, `index.js:134–167`) is registered **after** the command handlers (`:80–96`); Telegraf's first-match semantics let `/ask`, `/price`, etc. execute in restricted topics (bot replies leak into them; only plain text is deleted). Register the guard first.
- **[CWA-41] creator-gateway:** Google `access_token` is embedded in the client-held HS256 JWT (`auth.ts:52–59`) — base64-readable by anyone holding it; any XSS/log capture yields a live Google API token. At minimum shorten its scope/expiry; better: keep provider tokens server-side.
- **[CWA-42] creator-gateway:** `jwt.verify` without `algorithms`: pinning (`middleware/auth.ts:17` — hardening, not a live break with current `jsonwebtoken`); also `ifrLockAddress` default is `mainnet-format` while `chainId` defaults to `11155111` (`config/index.ts:4–5`) — a fresh deployment silently checks the wrong chain.

Informational

- **[CWA-43] Benefits backend hygiene:** `GET /api/businesses/:id[/rules|/products]` lack the rate limiter their siblings have; `GET /api/sessions/:id` is unauthenticated **and performs a DB write** (auto-expire) — a write-on-GET pattern worth removing (session IDs are cuids, so practical exposure is limited to observed ones).
- **[CWA-44] Telegram operational notes:** verified tier is cached until bot restart (a user who unlocks keeps Vote-topic access meanwhile — `reverifyFromMap` only re-checks unverified users); the wallet map persists to `/tmp/ifr_wallet_map.json` (world-readable on shared hosts; a local write there survives restarts via `autoRestoreAll`).
- **[CWA-45] Telegram content trust:** `/ask` interpolates the LLM answer into a Markdown group message (prompt-injection → crafted links in the group; cost bounded by `3/h/user + 50/h global` (500-char truncation); channel→community auto-sync reposts and pins whatever the bot sees as `channel_post` — safe only while the bot is admin in exactly the official channel.

3. Positive patterns (verified, do not regress)

- **Redemption atomicity (Benefits):** conditional `updateMany({status: 'APPROVED', expiresAt: gt})` inside a transaction that first takes the writer lock; daily/monthly caps counted under the same lock (`sessionService.ts:714–776`); pass binding claims via `updateMany({status: 'OPEN'})`.
- **Challenges are single-use and atomically consumed** (compare-and-set on `consumedAt: null`) across seller, customer-history, and customer-pass flows.
- **IDOR closed:** seller mutations re-verify ownership inside the transaction; customers list only their own history; passes require a 256-bit control token stored as SHA-256.
- **Fail-closed rate limiting:** Redis outage → 503, boot refused without Redis when configured, multi-replica+SQLite topology rejected at config parse.
- **Validation:** zod `.strict()` schemas, address regexes, HTTPS-only credential-free URL policy, canonical slugs with reserved-word guards; all raw queries parameterized.
- **On-chain checks:** eligibility re-verifies bytecode, `chainId` and token address at a single block tag; reward verification requires governance alignment + beneficiary match.

- **Admin surface:** static bearer with constant-time compare, ≥ 32 -char + placeholder blocklist at boot, per-IP 60/h limiter, pseudonymized audit log on every mutation.
- **SDK:** no secrets, address validation on all entry points, raw-bigint math, and the benefits client byte-for-byte validates the server challenge template before signing (anti-phishing). Dashboards contain no keys (public Alchemy demo fallback only). The wallet prototype is deliberately capability-free (Sepolia-only, no transfer/approval paths).

4. Prioritized recommendations

1. **creator-gateway (CWA-28/29/30):** remove `/auth/wallet`; require SIWE before any JWT is minted; bind Google OAuth to an existing SIWE session; add `domain` to SIWE verify here and in points-backend (CWA-04) in one sweep. Until then: do not expose the service publicly.
2. **Telegram verify (CWA-31...34):** DM-only codes, CSPRNG nonces, rate limits, wallet uniqueness, masked/absent wallet in status responses.
3. **Benefits:** domain/chain-bind the seller message (CWA-36), close the read-replay window where cheap (CWA-35), move the session auto-expire write off the GET path (CWA-43).
4. **Telegram hygiene:** guard registration order (CWA-40), strict origin parsing (CWA-39), fail-closed config (CWA-38), persist the wallet map outside `/tmp` with 0600 perms (CWA-44).

5. Reviewed vs. skipped (honest coverage)

Reviewed (all security-relevant logic read at the pinned commit): benefits backend — `index.ts`, `config.ts`, all middleware, routes `seller/customer/passses/admin/businesses/ sessions/attest`, and the services listed in the findings (incl. `sellerAuth`, `sellerAuthorizationChallenge(s/Actions)`, `rateLimitInfrastructure`, `rateLimitTopology`, `adminSecretPolicy`, `adminAudit`, `sessionService`, `customerPassService`, `customerHistoryAuth`, `sellerAccess`, `sellerLimits`, `sellerLimitPolicy`, `businessLifecycle`, `businessProfile`, `businessSlug`, `productPrice`, `lockSource`, `ifrLockService`, `rewardService`), Prisma schema. Telegram bot — `index.js` incl. the verify API, all admin commands, `verify.js`, `handlers/verification.js`, `verificationStore.js`, `onChainReader.js`, `skywalker.js`, `moderation.js`, `communityLinks.js`, `bootstrapListener.js`, `governanceNotifier.js`, `Dockerfile`, `railway.json`. creator-gateway — all of `src/`. SDK — `index.ts`, `benefits.ts`. dashboards — `config/hooks/key` components. benefits-wallet-prototype — all of `src/`.

Skipped: benefits-network frontend (Next.js client code beyond the skimmed libs), builder app, indexer stubs, `patches/` archive. Not executed anywhere: no dynamic testing, no live deployment probing — every finding is static-analysis-based; "works as-coded" means the code path exists as written, not that it was exploited.

6. Limitations & disclaimer

Static review at the stated commit; no guarantee of completeness; not financial or legal advice. The two High findings' real-world impact depends on whether creator-gateway is publicly deployed — that could not be determined read-only. Verify deployment status first, then prioritize accordingly.

IFR Protocol — Web3 Integration Audit (wallets, wiring, interfaces)

Auditor: Collateral Web3 Open Audits **Client:** NeaBouli / Inferno Protocol (ifrunit.tech) **Date:** 2026-09-14
Baselines: repo NeaBouli/inferno @ eb538a355001b042b343bfb19221af2407a96e63 · mainnet block 25971217 · Playwright @repo version (chromium-headless-shell v1228) · Foundry 1.7.1 **Method skill:** web3-integration-audit (function & wiring, proven) **Companion documents:** full-scope audit (CWA-01...24, PR #90), contract deep audit + surfaces supplement (CWA-25...45, PR #91) **Report version:** 1.0

This audit answers a different question than the security reviews: **is everything correctly wired, and does it actually work?** Scope: the /web3/ dApp (identical on apex and web3.ifrunit.tech), the shop.ifrunit.tech Benefits frontend, the Benefits wallet prototype, the wiki verify flow, and the API wiring that serves them.

Verdict: WORKS — with findings. Every contract call path in the dApp is selector-exact, the repo's own browser suites pass 44/44 locally, chain enforcement is fail-closed, and the write flows use exact-amount approvals with confirmation-gated success messages. Three findings: a mislabeled loan-struct decoder in the copilot API (CWA-46), a runtime third-party script dependency on the signing page (CWA-47), and a wrong-by-default offline fallback in the state reader (CWA-48).

1. Executive summary

The wallet and contract layer is well built: EIP-6963 discovery with legacy fallback, deliberate WalletConnect v2 setup (pinned version, minimal method surface, mainnet-only), exact-amount approvals, `tx.wait()` before any success claim, consistent 9-decimal handling everywhere, and the lending borrow UI is correctly disabled while the price oracle is unset (matches chain state). The repo's Playwright suites (20 wallet-connect + 24 web3-write tests) reproduce green in a clean environment.

The findings are integration-accuracy issues, not security holes: one API decodes the loan struct wrong (harmless today — zero loans — but wrong as-coded), one runtime CDN dependency weakens the otherwise fully self-hosted signing page, and one fallback path presents wrong bootstrap state instead of an honest error when RPCs fail.

Findings: 0 Critical · 0 High · 1 Medium · 2 Low · 2 Informational (CWA-46 ... CWA-50)

2. Touchpoint inventory

/web3/ dApp (static page; identical bytes on apex and web3.ifrunit.tech — verified):

- Scripts: vendored ethers-6.17.0.umd.min.js (self-hosted, integrity-positive), then web3-wallet-core.js (IFRWallet v4.3) and assets/ifr-state.js (IFRState).
- Wallet layer: EIP-6963 announceProvider / requestProvider + legacy multi-provider handling; WalletConnect v2 via @walletconnect/ethereum-provider@2.17.3 with methods: ["eth_sendTransaction", "personal_sign"] only, chains: [1], real project ID, QR on desktop, URI handoff + 30 s recovery polling on mobile; MetaMask deeplink helper; EIP-747 wallet_watchAsset with canonical token metadata (decimals 9).
- Contracts called: InfernoToken, IFRLock, CommitmentVault, LendingVault (write flows); BootstrapVaultV3 (status reads).

- Signatures: none in the dApp itself (no `personal_sign` usage found on a write path — the WC method list is just capability declaration).
- Reads: `eth.llamapc.com` fallback provider; 60 s auto-refresh in `IFRState`.

shop.ifrunit.tech (Benefits frontend, Next.js 15 + wagmi):

- Connectors: `injected()`, `injected({target:'phantom'})`, `coinbaseWallet`, and `walletConnect` **only when `NEXT_PUBLIC_WALLETCONNECT_PROJECT_ID` is set** — a missing ID hides the connector instead of presenting a broken one (matches the "not configured yet" docs; correct gating).
- Chains: mainnet by default, Sepolia only via `NEXT_PUBLIC_CHAIN_ID`; RPC overridable via env with `publicnode` fallback.
- Contracts: token / `IFRLock` / `CommitmentVault` with minimal ABIs (verified below), `IFR_DECIMALS = 9` enforced in `formatIFR` via bigint arithmetic (no float rounding of base units).
- Signature flows (server-challenge based) live in the backend and were reviewed in the surfaces supplement (CWA-35/36); the SDK's benefits client byte-for-byte validates the challenge template before signing.

Benefits wallet prototype: deliberately capability-free (Sepolia-only, no transfer/approval paths, CDP export with `skipMfa={false}`) — nothing to mis-wire.

Wiki verify page: client-side EIP-191 signing against the Telegram verify API; the crypto check is correct server-side (recovered address compared to claimed wallet); the server-side weaknesses are CWA-31...34.

3. Wiring verification (selector-precise)

Method: canonical 4-byte selectors computed from the compiled artifacts at the pinned commit vs. every human-readable ABI fragment embedded in the dApp page (script: `evidence/selector-check.mjs`).

Result: 31/31 MATCH, 0 problems (`evidence/selector-check.txt`) — every function the dApp calls exists on the deployed contracts with the exact signature used:

- `InfernoToken`: `balanceOf` `allowance` `approve`
- `IFRLock`: `lock` `lockWithType` `unlock` `lockedBalance` `lockInfo`
- `CommitmentVault`: `lock(uint256,uint8,uint256,uint256)` `unlock(address,uint256)` `lockedBalance` `getTrancheCount` `getTranche` `isConditionMet` `priceOracle`
- `LendingVault`: `createOffer` `increaseOffer` `withdrawOffer` `borrow` `repay` `topUpCollateral` `hasOffer` `lenderOfferIndex` `ifrPriceWei` `getRequiredCollateral` `getInterestRate` `getOfferCount` `getOffer` `getLoanCount` `getLoan` `calculateInterest` `getCollateralRatio`

Struct decoding order verified manually against the contracts: `Tranche` (`amount`, `cType`, `unlockTime`, `p0Multiplier`, `unlocked`, `conditionMetAt`) and `Loan` (`borrower`, `offerId`, `ifrAmount`, `ethCollateral`, `startTime`, `duration`, `monthlyRateBps`, `active`) match the page's tuple definitions exactly; the shop frontend's `getTranches` components match as well. Bootstrap status tuple (`active`, `_finalised`, `totalETH`, `timeRemaining`, `contributorCount`) matches `getBootstrapStatus`.

Write-flow mechanics verified in source (`docs/web3/index.html` script block):

- **Lock:** reads `balanceOf` + `allowance` first, honest insufficient-balance error naming the exact amounts, **exact-amount approve only when allowance insufficient**, waits for the approval receipt, then `lock/lockWithType(amount, bytes32(lockType))`, success text only after `tx.wait()`. Unlock path:

`unlock()` + confirmation, no re-approval needed (suite "existing IFRLock balance can be unlocked without another approval" passes).

- **Commitment:** `approve(totalAmount)` then per-tranche `lock(amount, cType, unlockTime, p0Multiplier)`; the UI's condition-type selector maps to the contract enum with correct ordinals (0=TIME_ONLY ... 3=TIME_AND_PRICE) — note the contract-side permanence risk for price types is CWA-03, and the page correctly does not hide that price conditions are unavailable.
- **Lending:** `createOffer / increaseOffer` with exact approve; `borrow(offerId, amount, durationDays, {value: collateral})` with collateral preview from `getRequiredCollateral` — and the whole borrow path is **transaction-disabled while `ifrPriceWei == 0`** (UI label "Disabled"; proven by the suite). `repay` approves principal + interest; `topUpCollateral` payable top-up.
- All amounts handled in base units via `parseUnits(..., 9) / formatUnits(..., 9)` — no float rounding of IFR values (the repo AGENTS.md rule is honored).

4. Configuration findings

Check	Result
dApp chain enforcement	mainnet enforced at connect (hex/decimal/zero-padded chain IDs all parsed; rejection throws <code>WRONG_NETWORK</code> before any state) — fail-closed, suite-proven
Chain-change handling	full page reload on <code>chainChanged</code> ; account change re-derives signer and re-reads state; disconnect cleans <code>localStorage</code>
dApp addresses	all 4 + bootstrap + pair match the verified deployment map (block 25971217)
Shop env defaults	mainnet-first (<code>NEXT_PUBLIC_CHAIN_ID</code>)
WalletConnect config	real project ID, mainnet-only, minimal methods — correct; project ID is public by design (Info: restrict allowed origins in the Reown dashboard)
RPC endpoints	llamarpc (dApp), publicnode fallback (shop/liquidity calculator); public RPCs see user IPs — disclosed in the wiki liquidity page
ethers.js delivery	self-hosted UMD build (v6.17.0) — the right pattern; contrast CWA-47

5. Dynamic execution results

- **Playwright suites at the pinned commit: 44/44 PASS (29.4 s)** — 20 wallet-connect + 24 web3-write tests (evidence/playwright-run.log). Environment note: the first run failed 40/44 purely because the repo-pinned Playwright browser build (chromium-headless-shell v1228) was not installed on this machine; after `npm install playwright chromium` all 44 passed. Nothing in the app caused the initial failures. Highlights among the proven behaviors: wrong-chain rejection fails closed *before* any connected state; WalletConnect-style numeric and zero-padded-hex mainnet IDs connect without false errors; EIP-6963 dedup; exact-approve + typed lock only on mainnet; IFR base units preserved through CommitmentVault/LendingVault writes; borrowing stays disabled at price 0; self-hosted ethers asset enforced; service-worker offline shell bounded.
- **Benefits wiring battery: 13/13 suites PASS** — the project's own `test:benefits-*` and routing/wiki suites (money math, service worker, offline shell, wallet asset, pass UI, discovery UI, wallet UI incl. negative scenarios, CSP gate, fullstack e2e with production Next.js build, bundle budget, doc consistency, surface routing, wiki head integrity). Environment notes (evidence/benefits-suites-exitcodes.txt, benefits-suites-round2.txt): the apps carry their own lockfiles, so a root-only `npm ci` leaves `apps/benefits-network/*/node_modules` empty and 8 suites fail on missing dependencies —

after per-app `npm ci` (the CI recipe) they pass; the CSP gate additionally requires a running frontend (`BENEFITS_BASE_URL`, default `127.0.0.1:3000`) and passes once it is started (CSP compatibility gate passed). No product failure was found in either round.

- Fork-level write simulation against an anvil mainnet fork was not needed for a verdict: the contract logic itself is fuzz-proven in the deep audit, and the dApp's flows are selector-exact with suite-proven UX. No live transactions were sent (read-only policy).

6. Findings

Medium

[CWA-46] copilot-api decodes the LendingVault `Loan` struct with a shifted ABI

- **Severity:** Medium (dormant today) · **Likelihood:** certain once loans exist · **Status:** Open
- **Component:** `apps/ai-copilot/server/index.ts:1571-1575` (`LV_LOAN_ABI`)
- **Description:** the API's `getLoan` ABI declares `tuple(address borrower, uint256 ifrAmount, uint256 ethCollateral, uint256 startTime, uint256 duration, uint256 monthlyRateBps, uint256 repaidAt, bool active)`. The deployed struct is `(borrower, offerId, ifrAmount, ethCollateral, startTime, duration, monthlyRateBps, active)`. The API fragment **omits `offerId` and invents `repaidAt`**, so every `uint256` field after `borrower` is read from the wrong slot: `ifrAmount` actually returns the offer ID, `ethCollateral` returns the IFR amount, `startTime` returns the collateral, and so on. Affects `/api/lending/loans/:address` and `/api/lending/health/:loanId` field values.
- **Impact:** today zero — `getLoanCount() = 0` on-chain (verified, block 25971217). From the first real loan onward, users and the AI copilot would see/serve wrong amounts and dates.
- **Evidence:** contract struct `contracts/vault/LendingVault.sol:51-60`; the dApp page's own (correct) tuple for contrast. Selector `0x504006ca` is unaffected — the bug is decoding-side.
- **Recommendation:** fix the ABI to the real struct order and add a decoding test against a known mainnet loan fixture before lending activation.

Low

[CWA-47] WalletConnect provider is imported from the `esm.sh` CDN at runtime

- **Severity:** Low · **Likelihood:** low · **Status:** Open
- **Component:** `docs/web3-wallet-core.js:47,283-346`
- **Description:** `import("https://esm.sh/@walletconnect/ethereum-provider@2.17.3")` executes third-party-hosted code at runtime on the page that handles signing sessions. The version is pinned and the choice is documented (jsdelivr bundling broke), but there is no integrity mechanism for ESM imports and no fallback; combined with the missing CSP on `web3.ifrunit.tech` (CWA-10), a compromised/unreachable `esm.sh` silently breaks or could alter the connect flow.
- **Recommendation:** self-host a bundled WC provider (same pattern as the vendored ethers UMD) or add a CSP `script-src` allowlist once headers ship on the nginx host.

[CWA-48] IFRState's last-resort fallback presents wrong bootstrap state as fact

- **Severity:** Low · **Likelihood:** low (requires both RPC paths failing) · **Status:** Open
- **Component:** `docs/assets/ifr-state.js:113-122`

- **Description:** if the aggregated and individual bootstrap reads both fail, the state object falls back to hardcoded defaults claiming `finalized: false` and `totalETHRaised: "0.0"` — while the chain truth (verified) is `finalised = true`, 0.03 ETH, 3 contributors. The failure direction is funds-safe (no false "claim now" affordance), but the UI states a wrong fact instead of saying "unknown".
- **Recommendation:** surface an explicit "status unavailable" state instead of defaults; if defaults are kept, at least set `finalized: true` and label the values as cached/unknown.

Informational

- **[CWA-49] WalletConnect project ID is hardcoded and public** (`web3-wallet-core.js:38`) — by design for client-side IDs, but the Reown dashboard should have origin restrictions set; not verifiable from outside.
- **[CWA-50] ifr-state reads cover token/lock/bootstrap only** — CommitmentVault/LendingVault numbers on the site come from the copilot API instead of direct reads; consistent with the architecture, but it makes CWA-46-class API decoding bugs user-visible. Prefer direct chain reads (the pattern exists in the liquidity calculator) for money-critical numbers.

7. Honest limitations

No real-device wallet testing (MetaMask/Trust/Coinbase mobile flows are mock-level only, via the project's own suites); the WalletConnect QR handshake was not executed end-to-end with a live relay; no write transactions on live networks (read-only policy); shop frontend reviewed at config/library level plus backend-covered flows, not pixel-level. The initial Playwright environment failure was tooling, not product — stated here because the report's credibility depends on that distinction.

— Collateral Web3 Open Audits, 2026-09-14

IFR Protocol — README Audit (claim-by-claim consistency review)

Auditor: Collateral Web3 Open Audits **Client:** NeaBouli / Inferno Protocol (ifrunit.tech) **Date:** 2026-09-14
Baselines: repo NeaBouli/inferno @ eb538a355001b042b343bfb19221af2407a96e63 · mainnet block 25971217 · web surfaces 2026-09-13/14 **Companion documents:** CWA full-scope audit (CWA-01...24), contract deep audit (CWA-25...27), surfaces supplement (CWA-28...45), Web3 integration audit (CWA-46...50)
Report version: 1.0

Scope: the repository's public face — README.md (371 lines). Every verifiable claim was checked against chain state (block-pinned reads), reproduced tool runs, source review, or link liveness. Verdicts: **Confirmed** / **Diverged** / **Overstated** / **Unverifiable-as-stated**.

Verdict: largely accurate, with 2 substantive divergences and 1 overstatement. The README's honesty patterns (dated snapshots, explicit "older internal inventory" disclaimers) are above industry norm and made this review faster — the fixes below are cheap because the framing is already honest.

Findings: 0 Critical · 0 High · 2 Medium · 1 Low · 3 Informational (CWA-51 ... CWA-56)

1. Claim-by-claim results

Branding, addresses, links

Claim	Verdict	Evidence
Token contract 0x77e9...A6e7B , LP pair 0xbE49...31A0	Confirmed	chain reads @ 25971217
Contracts table: 14 protocol contracts + LP + deprecated V1, all addresses	Confirmed	every address matches the verified deployment map; all dual-explorer verified
Bootstrap FINALIZED 05.06.2026; LP retained in vault, no withdrawal function, Team.Finance disabled	Confirmed	getBootstrapStatus() = (false, true, 0.03 ETH, 0, 3); lpLockId=0 ; LP balance = totalSupply - 1000 wei
51 internal docs/ links + 3 relative files exist; external spot-checks (token-list.json, wiki, raw asset) return 200	Confirmed	evidence/readme-link-check.txt
Safe section: Treasury 0x5ad6...cE3b , threshold 3-of-5, 5 signers	Confirmed	getThreshold()=3 , getOwners()=5 documented addresses — on all three Safes

Token economics

Claim	Verdict	Evidence
2.5% burn per transfer (2% + 0.5%), 1% pool fee, 5% hard cap	Confirmed	senderBurnBps=200, recipientBurnBps=50, poolFeeBps=100 , cap enforced in setFeeRates (fuzz-proven)
Decimals 9, genesis 1B, no mint function	Confirmed	source + chain

Claim	Verdict	Evidence
"Current Supply ~997.67M (verified 22 August 2026)", "Burned ~2.33M (22 August 2026)"	Confirmed as dated snapshot	honestly date-labeled; current chain truth 997,571,140.02 / 2,428,859.98 (13.09) — fine as a dated figure
Team vesting: 12m cliff, 36m linear, "0 tokens available before March 2027"	Confirmed	start=1772670647 (2026-03-05), released=0, beneficiary immutable
Liquidity Reserve: lock ended 01.09.2026, 200M held, 50M/90d cap, no withdrawals	Confirmed	chain reads (lockEnd=1788222659, balance 200M, totalWithdrawn=0)

Governance & security statements

Claim	Verdict	Evidence
"governed by a 48-hour timelock ... Gnosis Safe multisig"	Mostly confirmed, exceptions named	delay=172,800 s; all owners/admins = Governance; but: all guardian roles = single EOA, setGuardian untimelocked (CWA-09), deployed Governance carries pre-W1 setOwner (CWA-25)
Slither paragraph (21 sources, 6 reviewed High baseline, 0 unreviewed High / 0 Critical)	Confirmed — reproduced locally	evidence/slither-run.log (exit 0)
Mythril paragraph (17 contracts, bounded, no signals)	Confirmed — reproduced locally	evidence/mythril-run.log (exit 0)
Internal audit 0 FAIL / 20 WARN / 1 fixed / 81 PASS (Skywalker)	Confirmed as document claim	note: W1/W3 are "fixed in source, not deployed" (CWA-25/26) — the README should not imply otherwise
App security review 12 findings (2 CRITICAL, 5 HIGH, all fixed)	Confirmed as document claim	spot consistency only
Bootstrap review 11/14 secure, 3 low, 0 critical	Confirmed as document claim	
"Governance lifecycle tested: propose → 48h → execute"	Confirmed	17 on-chain proposals, spot-decoded #15/#16 match the public log
"Full Sepolia testnet deployment with verified contracts"	Confirmed per docs	spot-check level
"No single entity controls the protocol"	Directionally true, needs the guardian caveat	one EOA holds pause/cancel everywhere (CWA-09)

2. Findings

Medium

[CWA-51] Allocation table: Treasury/Community custody rows diverge from chain state

- **Where:** README "Token Allocation" table (Treasury and Community & Grants rows)
- **Claim:** Treasury 150M held by "Gnosis Safe multisig (0x5ad6193...)"; Community "57.9M received ... 50M → Bootstrap. ~7.9M operational reserve".

- **Chain truth (block 25971217):** Treasury Safe holds **0 IFR**; Community Safe holds 7.9M (matches); the **LP Reserve Safe holds 400.6M** — i.e. the 400M DEX allocation plus what appears to be the aggregated Treasury/Community portions, but the table does not say so for those rows (only the DEX row names the LP Reserve Safe). The "50M → Bootstrap" wording is also not the documented flow (bootstrap funding came from LiquidityReserve).
- **Impact:** a reader verifying the README against Etherscan finds an empty Treasury Safe and no explanation — unnecessary FUD material against a project that is actually transparent.
- **Recommendation:** add custody notes per row ("currently aggregated in LP Reserve Safe 0x5D93..., see lp-strategy page") or regenerate the table from chain state; fix the "50M → Bootstrap" phrasing to the actual destination.

[CWA-52] "BuybackVault and BurnReserve accumulate from the 1% protocol pool fee" — not true on-chain

- **Where:** README line following the allocation table.
- **Chain truth:** the 1% pool fee is IFR-denominated and routes to **FeeRouterV1**, which has no withdrawal function — 371,543 IFR are stranded there; BuybackVault and BurnReserve each hold **0** (verified at block 25971217). This sentence is the README-level root of the flywheel misdescription tracked as CWA-02.
- **Recommendation:** same decision as CWA-02 (sink vs. re-point `poolFeeReceiver`), then align this sentence, FEE_DESIGN.md, transparency page and press-kit in one sweep.

Low

[CWA-53] AI Copilot "Automatic seed phrase / private key detection" is overstated

- **Where:** README "AI Copilot" section.
- **Reality:** the copilot server and widget (reviewed in full, `apps/ai-copilot/server/index.ts`) contain a static "never share keys" disclaimer and prompt-level safety rules, but **no automatic seed-phrase/private-key detector** (no pattern check on inbound messages or outbound rendering). The widget itself renders chat as text (safe), and the claim's intent is good — but "automatic detection" describes code that does not exist.
- **Recommendation:** either implement a two-line detector (regex for 12/24-word sequences / `0x[0-9a-f]{64}` on the inbound message → canned warning), or reword to "explicit safety warnings and knowledge-bounded answers".

Informational

- **[CWA-54] Test tables are stale but correctly disclaimed** — "Protocol Tests 521" / "Browser / WalletConnect 12" vs. reproduced reality (644 contract + 44 browser tests green at the pinned commit). The inline disclaimer ("older internal inventory... must not be read as a current total") is exactly the right pattern; extending `update-stats.yml` to regenerate these numbers would close the gap permanently.
- **[CWA-55] "17 documented on-chain components"** — the counted list is not published anywhere; the README's own contract table lists 16 rows. Publish the counted set once (token + 13 protocol + LP + safes? — pick a definition) and the number becomes self-verifying.
- **[CWA-56] "Integrated Builder Products" table vs. empty on-chain registry** — `getActiveBuilders()` = `[]` (block 25971217). The listed products may be real integrations in progress, but the README does not distinguish "product-level integration" from "registered on-chain builder". One clarifying sentence prevents a wrong conclusion either way.

3. What the README gets right (keep doing)

- Dated, labeled snapshots instead of silent live claims ("verified 22 August 2026").
- Explicit "older inventory" disclaimer above the test tables.
- Honest placeholder labeling for `admin-console / investor-web`; "not used in production" for the wallet prototype; "npm publication pending" for the SDK.
- Bootstrap/LP custody described with the actual mechanism (no withdrawal function) rather than marketing shorthand.
- 51/51 internal links resolve; external links sampled are live.

4. Limitations

Point-in-time at the stated baselines. "Diverged" verdicts are about README-vs-chain accuracy, not accusations — every divergence found has an innocent operational explanation in the project record; the issue is only that the README does not yet tell that story.

— Collateral Web3 Open Audits, 2026-09-14

IFR Protocol — Content Coherence Audit (landing + wiki + shop copy)

Auditor: Collateral Web3 Open Audits **Client:** NeaBouli / Inferno Protocol (ifrunit.tech) **Date:** 2026-09-14
Baselines: repo NeaBouli/inferno @ e8cf1ece8c40c0fa19f18344fe25593304ef5c26 (main HEAD; all audited content pages byte-identical to audit baseline eb538a35 — the only merges since are the audit documents themselves) · mainnet block **25974493** (fresh reads; evidence/content-audit/chain-reads-content.txt)
Companion documents: CWA full-scope audit (CWA-01...24), contract deep audit (CWA-25...27), surfaces supplement (CWA-28...45), Web3 integration audit (CWA-46...50), README audit (CWA-51...56) **Method skill:** web3-content-coherence-audit (this audit is its reference application) **Report version:** 1.0

Scope: every public content page served from the repo — landing docs/index.html, docs/builder.html, the wallet dApp copy of docs/web3/index.html, all **36** wiki pages (docs/wiki/*.html), and the customer-facing copy of the Benefits Network PWA (apps/benefits-network/frontend, sitemap routes / /guide /scan /privacy /support plus global UI strings and llms.txt). ~500 checkable claims extracted into a ledger, each verified against block-pinned chain reads, source at the pinned commit, or CI records. Prior audit register (CWA-01...56) deduplicated: still-open known instances are marked, not re-registered.

Verdict: the engineering claims are largely true; the editorial layer has drifted. Every contract address on every page is correct; every fee/lock/vesting parameter that matters checks out against chain; the shop copy is fully accurate (83/83 claims verified). The defects concentrate where pages describe *history* (proposal logs, funding tables) and *mechanisms not yet live* (buyback, vouchers, lending yield) — plus one systematic wrong-network address column.

Findings: 0 Critical · 0 High · 10 Medium · 7 Low · 1 Informational (CWA-57 ... CWA-74)

No finding is live-exploitable or moves funds; the audit target here is the *public record*. Nothing in this report changes any CWA-01...56 severity. Two proposed agent findings were **dropped after lead verification** (see §7 honesty notes) — the wiki's "Deployer holds 0 IFR" is true today, and PartnerVault's partners() getter exists (the doc errors are narrower than proposed).

1. Surface inventory

39 repo-served pages + shop PWA copy. Full table with byte sizes: evidence/content-audit/page-inventory.txt. The three wiki pages added since the 2026-06-16 internal content audit (community-signer-expansion.html, liquidity.html, reputation.html) are included; the prior audit's fixed-claims list was re-verified and holds.

2. Claims ledger summary

Class	Extracted	Verified OK	Contradicted/Stale (new)	Known (CWA-01...56)	No-source
Landing + dApp pages	~165	131	9	22	3
Wiki 36 pages	~250	176	38	24	12
Shop PWA copy	83	83	0	0	0
Total	~500	390	47 instances → 18 findings	46	15

(Per-page ledgers with verbatim quotes: agent extraction logs, retained in `evidence/content-audit/`; every finding quote below was re-verified line-by-line by the lead — `evidence/content-audit/page-checks.txt`.)

3. Cross-page consistency matrix (repeated facts, all located variants)

Fact	Values found across pages	Truth (evidence)
Contract/component count	13 (multisig:869) · 14 (deployment:612, wallet-guide:797) · 16 (mainnet-checklist:632) · 17 (10+ pages) · "17 = 13+3" (open-audit:687, sums to 16)	14 contracts + 3 Safes (contract map, chain) — CWA-55 registered; the "13" and "13+3" variants are new
Sepolia contract count	17 (testnet meta) · 14 (testnet body) · 13+3 (testnet JSON-LD) · 16 (roadmap:621)	12 entries documented (contract-map:64); page also claims "10 proposals executed" vs its own table's 6 (CWA-70)
Wiki page count	33 (2026-06 audit) · 35 (one-pager:710, roadmap:648) · 36 (wiki/index meta ✓)	36 files at pinned commit
Tier thresholds	1,000/2,500/5,000/10,000 (lock-mechanism, benefits app) · 1,000/5,000/25,000/100,000 (press-kit) · 1,000/5,000/25,000 (integration) · 500/2,000/10,000 (TierModule + <code>/api/ifr/check</code>) · builder slider 100–10,000	CWA-20 registered; benefits-app internal tension (500 template vs 1,000 "typical") newly noted in CWA-74
P0 denomination	300,000,000 wei (faq:582, governance:1159, lp-strategy:841 ✓) · "0.030 ETH / 200,000,000 IFR" (commitment-vault:1076 ✗) · \$0.0000002 USD (ecosystem:1125, lp-strategy:825)	300,000,000 wei (Proposal #16, chain); USD figure implies ETH ≈ \$667, conflicts with pages using ~\$2,000
LP phase split	100/100/100M (tokenomics:1134) · 150/100/150M (protocol-plan:1045) · 100/150M (transparency:1131) · in-page 150M bar vs 100M text (protocol-plan:1045-1052)	Executed Phase 1 = 100M + 0.030 ETH (chain); claims-register #4 known; protocol-plan's bar/text split is new (CWA-69)
LP pool IFR content	100M initial ✓ (bootstrap:837) · ~195M (contracts:2137/2180 ✗) · 36.128M (bootstrap:1433, dated 16.06)	11,899,172 IFR @ 25974493 — buys drained the pool's IFR side; contracts.html is wrong in <i>both</i> directions
Buyback status	"fully active  " (contracts:2420, roadmap:712, tokenomics:1176) · "Inactive — activates Phase 3" (wallet-guide:981) · "ecosystem buyback into PartnerVault" (protocol-plan:1073)	Controller + vaults hold 0 ETH/0 IFR ; no buyback ever executed (CWA-02 family; CWA-59 for the invented mechanism)
Governance proposals	"#0–#9 executed" (wallet-guide:836) · "7 executed" (protocol-plan:937) · #1/#3 "executed" (deployment:911/923) · #6 dual identity (transparency:1359 vs :1505)	Chain-decoded @ 25974493: #1/#3 cancelled (targets Vesting/BootstrapVaultV3), #6 = setPoolFeeReceiver(FeeRouterV1), executed (CWA-61)
Multisig phase history	Phase 0 "Single EOA" (governance:1604) · Phase 0 "Genesis 1-of-2" + 2c "2-of-5" (faq:884, multisig:618)	Two irreconcilable histories on different pages (CWA-69)
EOA→Safe migration date	06.03.2026 (transparency:822) · 07.03.2026 (transparency:1021/1030, wallet-guide:768/785)	Same event, two dates, one page self-conflicted (CWA-69)

Fact	Values found across pages	Truth (evidence)
Treasury→Bootstrap amount	~150M (transparency:1042, roadmap:681, tokenomics:996) · 144.75M (transparency:1401, wallet-guide:690)	Both on the <i>same</i> transparency page; the 144.75M figure also contradicts the page's "feeExempt (zero burn)" note for that transfer (CWA-72 family note)
Deployer feeExempt	"explicitly revoked" (tokenomics:1224/1242/1309) · "Active since 18.03.2026" (wallet-guide:656, transparency prop #7)	Chain: feeExempt(Deployer) = true @ 25974493 → tokenomics wrong (CWA-63)
Burned supply	"2.3M" (index:2309) · "2.33M" (index:2757) · ~2.43M actual	1e9 – totalSupply = 2,428,859.98 IFR @ 25974493 (CWA-71)
Test counts	27 (bootstrap:1048) · 544 (historical, disclaimed ✓) · 642 (index:1694) · 644+30+36 (faq:979 ✓)	CWA-20/54 known; bootstrap's 27 vs actual 30 <code>it()</code> new (CWA-70)
DAO phase labeling	"Phase 4 — Verified" (open-audit:690) · "Phase 4 NEXT" (one-pager:718) · "Phase 5" (security:1226) · Phase 3 = multisig Q3–Q4 2026 (press-kit:847) while 3-of-5 active since 15.03.2026	No RatVoting/ForumVoting/IFRSpamProtection exists in <code>contracts/</code> (grep: 0) — "Verified" impossible (CWA-69)

4. Findings

Medium

[CWA-57] The advertised 15-bps points voucher can never execute on the deployed FeeRouter

- **Severity:** Medium · **Status:** Open — works as coded, promise impossible
- **Pages:** docs/wiki/agent.html:865 ("Discount: 15 bps"), :898–899 ("Fee drops to 0 when discount exceeds fee"); docs/wiki/tokenomics.html:1501–1502 ("100 points → 15 bps discount"); docs/wiki/security.html:1271 ("Discount > Fee → Clamp: max(fee – discount, 0)")
- **Truth:** FeeRouterV1.sol:130 — `require(voucher.discountBps <= protocolFeeBps, "Discount exceeds fee")` : `discount > fee` **reverts**; the clamp branch at :151–154 is unreachable via the voucher path. With `protocolFeeBps = 5` (:23, chain-confirmed), the documented 15-bps voucher reverts on every use. `isVoucherValid` would return ("Discount too high") (:170).
- **Impact:** the points→voucher benefit advertised on three pages is structurally undeliverable at current parameters. Dormant today (points system not live), so no user harm — but the docs describe a mechanism the deployed contract forbids. Adjacent to CWA-18 (maxUses dead code), distinct defect.
- **Fix direction:** either document "voucher ≤ 5 bps at current fee" and cap issuance, or note the intended `discountBps ≤ fee` contract rule on all three pages; security.html's clamp row should describe the revert.

[CWA-58] Lending yield described with wrong asset and wrong destination

- **Severity:** Medium · **Status:** Open
- **Pages:** docs/wiki/lending-vault.html:834 — "Interest is split: 50% → Contributor (ETH yield), 50% → Uniswap LP (pool becomes deeper and more stable)"; docs/wiki/commitment-vault.html:897 — "Earns ETH yield via LendingVault during the lock phase" (present tense).

- **Truth:** `LendingVault.sol:228-246` — interest accrues in **IFR**, split 50% lender / 50% `protocolFeeReceiver` (unset on mainnet, CWA-08); no Uniswap LP route exists; and `totalLent = 0` @ 25974493 — no yield has ever accrued. `lending-vault.html` *itself* states the correct mechanism at :951 — the page contradicts itself.
- **Fix direction:** correct both sentences to the IFR 50/50 lender/protocol split; drop the present-tense yield claim until `totalLent > 0`.

[CWA-59] `protocol-plan.html` documents a buyback/refill mechanism that does not exist

- **Severity:** Medium · **Status:** Open
- **Page:** `docs/wiki/protocol-plan.html` — :1073-1075 "the protocol automatically reinvests accumulated fee IFR back into the builder ecosystem via the PartnerVault. It is not a market buyback with ETH"; :1103-1124 "PartnerVault $\geq$ 85% (34M) $\rightarrow$ auto-refill to 90% ... Overflow > 2M IFR $\rightarrow$ Treasury ... 70% Operating Pool / 30% SOS Reserve ... Community Vote (12 days)"; :1265 "Auto-rebalancing contract for PartnerVault refill + overflow routing. Deployed after LP launch."
- **Truth (code):** `BuybackController` does `swapExactETHForTokens` — an ETH $\rightarrow$ IFR **market buyback**, 50% $\rightarrow$ `burnReserve`, 50% $\rightarrow$ LP via `lpReceiver` (contract source; contract-map §105-109). No PartnerVault refill path, no 70/30 split, no SOS reserve, no 12-day vote exists in any contract (grep across `contracts/`). The controller was deployed 14.04.2026 — **before** the LP launch (05.06.2026), not after. Chain: `BuybackVault` 0 IFR, `BurnReserve` 0 IFR, controller 0 ETH @ 25974493.
- **Impact:** this is the CWA-02/52 flywheel family, but worse than drift: the page specifies parameters (85%/90%/2M/70/30/12 days) for machinery that was never built. Readers evaluating the token's value-accrual story get a fabricated mechanism.
- **Fix direction:** rewrite the section to the deployed design (or label it as unbuilt Phase-3 design with a visible "not implemented" banner).

[CWA-60] `contracts.html` "Sepolia" column links four mainnet contract addresses

- **Severity:** Medium · **Status:** Open
- **Page:** `docs/wiki/contracts.html` — :2299 `BuilderRegistry` "Sepolia" = `0x77e9...A6e7B` (mainnet `InfernoToken`); :2332 `CommitmentVault` "Sepolia" = `0xc43d...D041` (mainnet `Governance`); :2362 `LendingVault` "Sepolia" = `0x7699...54Eb` (mainnet `IFRLock`); :2394 `BuybackController` "Sepolia" = `0xaA14...6fCF` (mainnet `BurnReserve`). All four are `sepolia.etherscan.io` links carrying mainnet addresses — systematic copy-paste.
- **Impact:** an integrator following the "Sepolia" reference lands on mainnet contracts; the testnet page has the correct Sepolia set. Wrong-network address presentation is the classic pretext for signing-against-the-wrong-deployment mistakes.
- **Fix direction:** replace with the documented Sepolia addresses (contract-map:64) or drop the column.

[CWA-61] Governance proposal logs misstate executed/cancelled status and actions

- **Severity:** Medium · **Status:** Open — chain-decoded @ block 25974493
- **Pages:** `docs/wiki/deployment.html:911` "Proposal #1 executed (setFeeExempt IFRLock)" and :923 "Proposal #3 executed (setFeeExempt new PartnerVault)"; `docs/wiki/transparency.html` gives proposal #6 two different actions: :1359 "setPoolFeeReceiver(FeeRouterV1)  " vs :1505 "setFeeExempt(BootstrapVaultV3, true)".

- **Truth (chain):** `proposals(1) → setFeeExempt(Vesting, true)`, **cancelled**; `proposals(3) → setFeeExempt(BootstrapVaultV3, true)`, **cancelled**; `proposals(6) → setPoolFeeReceiver(FeeRouterV1)`, **executed**. So `deployment.html` is wrong on status *and* target for both rows, and transparency's table row for #6 is wrong (its :1359 narrative is right). Related link rot: `governance.html:1042/1051/1060` use 8-hex-char placeholders (`etherscan.io/tx/0xdb1590bf`, `0x1c885fbe`, `0x89eebca7`) as full TX hrefs — guaranteed 404s; same truncation pattern in `bootstrap.html:1230-1241` and `mainnet-checklist.html:1032/1036`.
- **Fix direction:** correct the #1/#3 rows to cancelled with the decoded targets; deduplicate #6; use full 66-char hashes or no link.

[CWA-62] Blanket "every change requires the 48h timelock" claims omit the live guardian exception

- **Severity:** Medium · **Status:** Open (docs side of W15/CWA-09)
- **Pages (verbatim instances):** `docs/index.html:2712` ("No instant admin changes — every action requires a 48-hour public delay"), :2829, `docs/wiki/index.html:586`, `fair-launch.html:727` ("no admin backdoors"), `faq.html:869` ("All changes require the full timelock delay"), `security.html:1181/1186`, `one-pager.html:700/725`, `multisig.html:778-779` ("Execute instantly — every change is public for 48 hours"), `transparency.html:1222`, `protocol-plan.html:930-937` ("No change can take effect instantly"), `wallet-guide.html:834`.
- **Truth:** `Governance.setGuardian` is `onlyOwner`, **not timelocked** (`Governance.sol:120`; W15 open, CWA-09) — the owner (TreasurySafe 3-of-5) can rotate the guardian instantly, and the guardian can cancel any proposal. Eleven pages assert universality without the exception. Note: the previously-cited second exception (BuybackController owner = Deployer EOA) no longer exists on-chain — owner = Governance @ 25974493 (see CWA-64); the guardian exception stands.
- **Fix direction:** one sentence wherever the blanket claim appears: "Exception: guardian rotation (`setGuardian`) executes without the timelock (W15, accepted)."

[CWA-63] `tokenomics.html` claims the Deployer fee exemption was revoked — chain says it is active

- **Severity:** Medium · **Status:** Open
- **Page:** `docs/wiki/tokenomics.html:1224` "Deployer's feeExempt status is explicitly revoked after distribution" (also :1242, :1309).
- **Truth (chain):** `feeExempt(0x6b36687b...ed67) = true` @ block 25974493; `transparency.html` proposal #7 ("`setFeeExempt(Deployer, true)`", 18.03.2026) and `wallet-guide.html:656` ("Active since 18.03.2026") are the correct pages. No revocation proposal exists in #8-#16.
- **Impact:** the `tokenomics` page — the canonical fee reference — denies an insider exemption that exists. Risk-understating (unlike CWA-64's safe-direction drift). The exemption itself is a documented accepted risk; the false *denial* is the finding.
- **Fix direction:** correct the three sentences; link the exemption table to the live state.

[CWA-64] Role/ownership reference drift across `contracts.html`, `multisig.html`, `mainnet-checklist.html`

- **Severity:** Low · **Status:** Open
- **Instances (each lead-verified):**

- `contracts.html:2402` — BuybackController "Owner: Deployer (0x6b36...)"; chain: `owner() = Governance (0xc43d...D041) @ 25974493`. Safe-direction drift. (Our own merged reference `tools/audit-skills/.../contract-map.md:28` carries the same stale row — self-flagged for correction.)
- `multisig.html:841-842` — "PartnerVault `admin()` Governance (immutable)"; code: `setAdmin` exists (PartnerVault.sol:391) — transferable, not immutable.
- `multisig.html:851-852` — "Vesting `guardian()` Deployer (revoke-only)"; code: Vesting has **no revoke function** — guardian can pause/unpause/transferGuardian (Vesting.sol:75-95).
- `multisig.html:741-742` — "Emergency pause (IFRLOCK)" listed under *what the Safe signers can do*; IFRLOCK's guardian is the Deployer EOA personally, not the Safe (CWA-09).
- `multisig.html:766-767` — timelock "Hardcoded in Governance contract. Not configurable."; code: `setDelay` exists, bounded 1h–30d (Governance.sol:112-116).
- `mainnet-checklist.html:969` — "FeeRouterV1 ... `feeCollector = Treasury`"; chain: `feeCollector = BuybackController` (Proposal #14, executed 18.04.2026 — which the same page lists as executed at :1167).
- `security.html:790-805` — W1 status "Multisig planned" / W3 "Fix in v3" vs the known-issues register's "fixed in source" state (and the CWA-25/26 not-deployed nuance).
- **Fix direction:** sync all role tables to the chain state; adopt one register as the single source.

[CWA-65] transparency.html supply table does not sum; three custody statements are false

- **Severity:** Medium · **Status:** Open
- **Page:** `docs/wiki/transparency.html` — the supply-distribution table (:1392–1416) lists LP Reserve Safe 400.6M + LiquidityReserve 200M + Vesting 150M + Treasury 0 + Community 7.9M + PartnerVault 40M + burned $\approx 2.4\text{M}$ $\rightarrow \approx 800.9\text{M}$ (80.1%), then a Total row claiming "**1,000,000,000 / 100%**". Missing rows: LP pair (11,899,172 IFR @ 25974493), CommitmentVault (47,952,477), LendingVault (52,155,441), FeeRouterV1 (371,544), user wallets (~184M). Separately on the same page: :1194 FeeRouterV1 "Pass-through — **holds no IFR**" (chain: 371,543.99 IFR stranded — the CWA-02 finding itself); :1201 IFRLOCK "No IFR held by contract itself (user balances tracked internally)" (code: `transferFrom(user, address(this), amount)` — IFRLOCK custodies all locked IFR, IFRLOCK.sol:117); :1222 "Immutable timelock hardcoded" (`setDelay` exists).
- **Impact:** the transparency page is the project's "verify everything" surface; a supply table that cannot sum and three contradicted custody statements undercut exactly the trust the page exists to build.
- **Fix direction:** recompute the table from chain (all balances above are in `evidence/content-audit/chain-reads-content.txt`); fix the three rows.

[CWA-66] lp-strategy.html's core arithmetic is wrong by orders of magnitude

- **Severity:** Medium · **Status:** Open
- **Page:** `docs/wiki/lp-strategy.html` — :865–869 "Milestone 1 ... Pool after: ~150M IFR + 1.5 ETH (\$3,000 depth) ... Buying 99% costs: ~\$30,000 instead of \$2,000"; :927–941 table "At P0 (\$0.0000002) 400M = \$80 / pool depth \$0.016 ... At P0x1,000 = \$80,000 / \$1,600,000" and "SAME 400M IFR = 100,000,000x more liquidity."
- **Truth (x·y=k, verifiable arithmetic):** draining 99% of a 150M-IFR/1.5-ETH pool costs **148.5 ETH $\approx$ \$297k** @ \$2k, not \$30k; from the actual 100M/0.03-ETH pool it costs 2.97 ETH $\approx$ \$5.9k, not \$2k. Matched-pool depth is $\approx 2\times$ one side's value (\$160 / \$160,000), not \$0.016 / \$1.6M — the table's ratio column is off by 100,000x, and the "100,000,000x" headline by 100,000x. Additionally the page implies ETH $\approx$ \$667 at :825 and uses ~\$2,000 at :865.

- **Impact:** the page argues a liquidity strategy to governance voters using numbers that do not survive arithmetic.
- **Fix direction:** recompute or remove the efficiency proof; keep the (correct) canonical facts: 100M + 0.030 ETH initial pool, 400.6M in the LP Reserve Safe.

[CWA-67] P0 denomination inconsistencies across pages

- **Severity:** Low · **Status:** Open
- **Instances:** `commitment-vault.html:1076` widget hint "P0 = 0.030 ETH / **200,000,000** IFR" — that ratio is 150,000,000 wei; the on-chain P0 is 300,000,000 wei (Proposal #16), and the code comment specifies the 100M denominator (`CommitmentVault.sol:70`). The same page states the correct value at `:964 / :1060`. USD-denominated "P0 = \$0.0000002" on `ecosystem.html:1125 / lp-strategy.html:825` implies $\text{ETH} \approx \$667$ and conflicts with the ~\$2,000 ETH assumption used on the same pages.
- **Fix direction:** wei everywhere (one canonical figure), USD only as dated examples.

[CWA-68] Developer-reference API drift (`integration.html`, `faq.html`, `lock-mechanism.html`, `governance.html`)

- **Severity:** Medium · **Status:** Open — every item lead-verified against source
- **Instances:**
 - `integration.html:1266–1272` documents `swapWithFee(adapter, data, tokenIn, amountIn, voucher);` actual: `swapWithFee(adapter, swapData, voucher, voucherSig, useVoucher)` (`FeeRouterV1.sol:90-96`).
 - `integration.html:1274–1284` documents `DiscountVoucher{user, discountBps, nonce, deadline, signature};` actual: `{user, discountBps, maxUses, expiry, nonce} (:42–48)`, and `isVoucherValid` returns `(bool, string) (:164)`, not `(bool)`.
 - `integration.html:1036` — `recordLockReward(partnerId, lockAmount);` actual: `recordLockReward(bytes32, uint256, address wallet)` (`PartnerVault.sol:265-269`).
 - `integration.html:1046–1051` — `partners(uint256)` returning `(address,uint256,...,bool,bool, string);` actual: `partners(bytes32)` returning `(address, 4×uint256, 3×uint32, bool, bool, uint8)` (`PartnerVault.sol:463-480`) — the getter exists (an earlier draft of this audit wrongly denied it), but key type and shape are wrong.
 - `integration.html:951–952` — "Ethers.js v5 (CommonJS)" while the same page's examples use v6 (`:756`, `:1068`) and the repo pins ethers 6.17.0.
 - `integration.html:1223` — `/auth/siwe/nonce` documented GET; implemented POST (`apps/points-backend/src/routes/auth.ts:20`).
 - `faq.html:1090` — lists `lock(uint256, string);` code has `lock(uint256)` and `lockWithType(uint256, bytes32)` (`IFRLock.sol:51,58`).
 - `faq.html:934` — "finalise() ... once the minimum ETH threshold is met"; no minimum-ETH threshold exists (`BootstrapVaultV3.sol:138-153` — time + IFR-balance checks only).
 - `lock-mechanism.html:788` — "lockedAt = block timestamp of the **first** lock"; code overwrites it on every lock (`IFRLock.sol:121`). `:835` — "stores a lockType for resolver metadata"; `lockType` is event-only, not stored (`:24–27`, `:125`).
 - `governance.html:943` — revert string documented as "Governance: too early"; actual: "too early" (`Governance.sol:89`).
- **Fix direction:** regenerate the API tables from the compiled ABIs (the repo already builds them — `/abi/`); one source of truth.

Low

[CWA-69] Status/phase incoherence across the wiki

- **Severity:** Low · **Status:** Open
- **Instances (lead-verified):** `open-audit.html:690` "Phase 4: Two-Chamber DAO, 48h timelock — **Verified**" (contracts don't exist; other pages say Phase 4 NEXT/planned); `open-audit.html:687` "17 on-chain components (13 + 3 Safes)" (sums to 16); `press-kit.html:847-854` schedules "multisig governance" for Phase 3 (Q3-Q4 2026) while 3-of-5 is active since 15.03.2026; `governance.html:1405` "Next: Multisig Governance" (live since 20.03.2026, same page :857); `governance.html:1105-1127` "Upcoming & Planned Proposals" + red "CRITICAL — must execute before first user lock/deposit!" on proposals #11/#12, both executed 06.04.2026; `ecosystem.html:1161` proposals #15/#16 "queued (ETA 08./09.06)" — executed; `protocol-plan.html:822/974/1227` "Bootstrap LIVE — accepting ETH" vs :994 "ENDED 05.06.2026" on one page; `wallet-guide.html:836` "Proposals: #0-#9 executed" (#11-#16 missing); `mainnet-checklist.html:1118-1127` FeeRouter setup all "Pending" (voucher signer is configured; 371,544 IFR have accrued); `protocol-plan.html:1045-1052` Phase-1 bar 150M vs executed 100M text; `roadmap/security/transparency` disagree whether DAO is Phase 4, Phase 5 or Milestone 5; multisig history: `governance.html:1604-1608` (Phase 0 "Single EOA") vs `faq.html:884` / `multisig.html:618-645` (Phase 0 "Genesis 1-of-2", extra 2-of-5 phase); EOA→Safe migration 06.03 vs 07.03 on `transparency.html` itself.
- **Fix direction:** one status register (single page) that others embed or link.

[CWA-70] Remaining count drift beyond CWA-20/54/55

- **Severity:** Low · **Status:** Open
- **Instances:** `testnet.html` four divergent Sepolia counts (17 meta / 14 body / 13+3 JSON-LD; `roadmap:621` says 16) plus "10 governance proposals were executed on Sepolia" vs its own table's 6 executed / 4 cancelled; `bootstrap.html:1048` "27 automated tests" (actual 30 top-level `it()` in `test/BootstrapVaultV3.test.js`); `one-pager.html:710` + `roadmap.html:648` "35 wiki pages" (36 at pinned commit; `wiki/index.html`'s own meta is correct); `multisig.html:869` "all 13 contracts verified" (14); `wallet-guide.html:635` "All 7 wallets" vs 8 wallet cards on the same page (:606-619 stats bar says 8).
- **Fix direction:** regenerate counts at build time where possible; at minimum fix the within-page contradictions.

[CWA-71] Landing-page stale fallbacks and mechanism summaries

- **Severity:** Low · **Status:** Open
- **Page:** `docs/index.html` — :2309 SVG fallback "997.7M supply · 2.3M burned" and :2757 "2.33M IFR Burned" (chain @ 25974493: **2,428,860 burned**; fallbacks disagree with each other); :2371 "FeeRouter 314.2K IFR fees held" (chain: 371,544); all three are `data-live-key` fallbacks — correct after JS loads, wrong for no-JS/preview contexts (link previews, crawlers, reader mode). :2266 "Irrevocable IFR lock ... Auto-unlock after 30 days" — the summary collapses two mechanisms: auto-unlock applies only 30 days *after a condition is met* (`CommitmentVault.sol:51,148-170`), and `TIME_ONLY` tranches are unlockable by design. :2974 "On-chain verified via `RatVoting.sol`" for a contract that exists nowhere (`grep contracts/` : 0 hits) — labeled "not live" one line up. :2 HTML comment "content baseline 2026-08-22" predates the page's own 26.08/01.09 content.
- **Fix direction:** update fallbacks to the current snapshot; rewrite the `CommitmentVault` summary; delete the `RatVoting` "verified" clause.

[CWA-72] bootstrap.html internal inconsistencies

- **Severity:** Low · **Status:** Open
- **Instances:** funding table (: 886–904) "Community Safe 60M sent / 57.9M received ... 50M Bootstrap + ~7.9M reserve" vs the same page's timeline rows (: 1230–1241) "Community Safe → Vault: 50M IFR" and parameter row (: 1173) "Treasury (150M) + Community (50M)" — three values for one fact. Footnote : 908 "Transfers were subject to the 3.5% fee-on-transfer ... Treasury sent 150M, vault received 150M" — impossible as written ($150M \times 0.965 = 144.75M$); the two Safes are fee-exempt on-chain, so the fee premise itself is doubtful. : 1406 example: "10% of the claim allocation" computed as ~19,475,000 IFR — 10% of 100M is 10,000,000. : 2231 second widget computes IFR-per-ETH from `parseUnits("200000000", 9)` while the page's wallet widget uses the verified 100M (: 1940) — the two on-page estimates diverge 2×. Timeline: contributions open 07.03 (: 1246) but "BootstrapVaultV3 deployed + verified 08.03" (: 1226) — coherent only via the deprecated V1, which the page mentions once in a struck-through table row (: 1037) without connecting it to the timeline.
- **Fix direction:** settle on 150M + 50M (the chain-consistent story), fix or delete the example and the second widget, one sentence on the V1→V3 migration.

[CWA-73] Mechanism/UX wording overstatements (assorted pages)

- **Severity:** Low · **Status:** Open
- **Instances:** `commitment-vault.html:854` "Once unlocked, stays unlocked — even if the price drops afterwards" — only true once `markConditionMet` has latched `conditionMetAt`; a plain `unlock()` re-evaluates the condition (`CommitmentVault.sol:148-170`). (Moot for price conditions on mainnet — CWA-03.) `docs/web3/index.html:1794` — "The hero is built with a replaceable Three.js layer": no Three.js exists in the page (grep: 1 hit — the sentence itself); the hero is a static image. `agent.html:798` — "every answer includes a Source: ... reference pointing to the specific documentation page or contract": the shipped widget shows only a static generic "Source: IFR_KNOWLEDGE" badge (`IFRCopilot.tsx:246`). `agent.html:799` — "the copilot **only** answers from IFR_KNOWLEDGE": the server additionally injects a live wiki fetch into the context (`server/index.ts:483`) — the knowledge-bound claim is stronger than the implementation (CWA-24 related). `open-audit.html:787` — a live-status JS snippet targets element ids absent from the page (never rendered; dead code). `builder.html:125` — static "85 SAFE" placeholder vs the same page's formula computing 90 (self-corrects on load). `tokenomics.html` — two sections numbered "7." (: 1419 , : 1465). Doc-version footers drift (v1.0 / v1.1 / v2.2 across pages).
- **Fix direction:** per-instance wording fixes; delete dead code.

Informational

[CWA-74] Notes and unverifiable-but-plausible claims (no defect)

- Migration arithmetic notes: Community Safe 60 – 50 – 7.9 leaves 2.1M unexplained on `index.html`'s flow (other spends possible); Treasury 150 – 144.75 = 5.25M unexplained in protocol-plan's text. `transparency.html:1208` "C2 50% lock" vs the figures it cites (20,156,940.95 of 33,333,333.33 = 60.5% — the 50% is LOCK_BPS of the *post-buy* balance; chain-consistent, wording confusing). Process claims with no repo source (48h PR review, LP-addition announcements, "all signers have hardware wallets" Pending vs multisig.html's policy-as-fact). Sepolia governance table rows (#0–#9) unverifiable read-only. The benefits app's "typical 1,000 IFR" (`WalletStatus.tsx:290`) vs its own 500-IFR "Welcome benefit"

template (SellerRuleBuilder.tsx:94) — both hedged, pulling opposite directions. vesting.html:984 (transferable guardian) is *correct per code* — the stale item here is our own register wording ("W18 guardian not rotatable"); reconcile in the register.

5. Link & structure integrity (repo-pinned, scripted)

Script: docs/community-audits/CWA_IFR_ContentCoherence_LinkCheck_2026-09-14.py (same convention as the CWA fuzz harness). Output: evidence/content-audit/link-check.txt.

- **39 pages scanned; 0 dead internal links; 0 orphan pages** — after resolving the site conventions (GitHub Pages root = docs/; https://ifrunit.tech/... treated as internal). The apparent 53 dead links in a naive run are root-absolute paths that resolve correctly against the site root — methodology note, not a defect.
- **3 dead in-page anchors**, all on docs/wiki/integration.html: #creator-gateway (×2, :1206–1207) and #e2e-flow (:1251) — link cards promise a "Creator Gateway Spec/Guide" and "E2E Flow Guide" section that does not exist (folded into CWA-68).
- 1,528 absolute ifrunit.tech references resolve; external hosts (276 Etherscan links etc.) not HEAD-checked en masse — spot checks OK; the truncated-hash Etherscan hrefs are covered in CWA-61.
- Live-vs-repo spot check: sampled pages served from GitHub Pages match the pinned commit (no deploy drift detected at audit time).

6. Verified strengths (checked and held)

- **Every contract address on every audited page matches the deployment map** — including all 13 rows on deployment.html, the landing's contract table, the dApp's dialogs, and the shop's contracts.ts constants. The only address-class defect is the wrong-network Sepolia column (CWA-60).
- **Shop copy: 83/83 claims verified** — profile limits (5/25), history TTL (10 min), page size (50), QR TTL bounds, single-redeem checkout, sessionStorage-only tokens, masked seller views, llms.txt statements (incl. "Sepolia prototype, not production") — all match the code. Cleanest surface of the audit.
- **liquidity.html: 14/14 claims verified against the calculator source** — the 0.997 factor, 60 s refresh, 3-minute expiry, and the capacity formula all match docs/assets/*.mjs.
- **C2's identical lock and offer amounts are TRUE on-chain** — offer(0) belongs to C2 (0x80f...B6958) with exactly 20,156,940.952845656 IFR @ 25974493, matching its CommitmentVault lock; the 50/50 LOCK_BPS design reconciles the identity. An agent-proposed "impossible" finding was dissolved by the chain read.
- **Core economics copy is correct wherever it matters:** fee split (200/50/100 bps), 5% cap, 9 decimals, no-mint, allocation buckets (sum = 100%), vesting arithmetic (4.17M/month, March 2027 cliff, March 2030 end), lending rate tables (200–2500 bps mapping), collateral thresholds (200/150/120), bootstrap parameters (0.01–2 ETH, 90 days), FAQ math examples, annualized-rate table, "82 questions" (counted: 82), wiki/index "36 pages" meta (counted: 36).
- **Honesty patterns worth keeping:** dated snapshots, "historical" disclaimers on the 544 figures, "internal audits ≠ professional certification" wording, and the seeded "fail-closed" disclosures on lending pages are accurate and made this audit faster.

7. Honesty notes (lead verification corrections)

- Agent proposal "governance.html 'Deployer holds 0 IFR' contradicts the ~21k IFR register entry" — **dropped:** `balanceOf(Deployer) = 0 @ 25974493`; the wiki is right, and the register note (AUDIT_SCOPE) is what aged. `transparency.html:1359`'s own narrative explains the ~21k accumulation window correctly.
- Agent proposal "PartnerVault has no `partners()` getter" — **narrowed:** the getter exists (`PartnerVault.sol:463`); the doc errors are the key type and return shape (CWA-68).
- Agent proposal "press-kit 'Anthropic API' claim unverifiable" — **dropped:** the copilot calls `api.anthropic.com/v1/messages` via `fetch` (`server/index.ts:488-492`); no SDK dependency is required for the claim to be true.
- `contracts.html:2402` staleness means our own merged `contract-map.md:28` (Deployer-EOA row) needs the same correction — flagged in CWA-64.

8. Limitations

- Live external links (276 Etherscan + others) not HEAD-checked en masse; repo-internal link integrity is complete (script above). TX hashes quoted on pages are unverifiable where pages truncate them (itself filed under CWA-61).
- Sepolia state verified only against the documented map; Sepolia proposal history is no-source (CWA-74).
- Dynamic shop routes (`/b /r /p /s`) are robots-excluded by design and were out of the copy scope; their wiring was covered by the Web3 integration audit (CWA-46...50).
- Chain state is a moving target: distribution figures are block-pinned to 25974493 and will drift (the LP pair's IFR side already moved 36.1M → 11.9M since the 16.06 snapshot).

9. Disclaimer

Content-coherence audit of public pages against public sources and chain state. Not a security audit of the underlying code (see companion documents) and not financial advice. All verification read-only; no transactions, no authenticated calls, no changes to the audited material. AI-assisted analysis; every finding quote and chain value was re-verified by the lead before inclusion.

IFR Protocol — AI Readiness Audit (copilot knowledge + AI anchors)

Auditor: Collateral Web3 Open Audits **Client:** NeaBouli / Inferno Protocol (ifrunit.tech) **Date:** 2026-09-14
Baselines: repo NeaBouli/inferno @ e8cf1ece8c40c0fa19f18344fe25593304ef5c26 · mainnet block 25974493 · live anchor checks 2026-09-14 (anonymous-browser GET/HEAD only) **Companion documents:** full-scope (CWA-01...24), deep audit (CWA-25...27), surfaces (CWA-28...45), integration (CWA-46...50), README (CWA-51...56), content coherence (CWA-57...74) **Method skill:** ai-readiness-audit (this audit is its reference application) **Report version:** 1.0

Scope, two halves: **(A) the IFR Copilot's knowledge** — can the bot answer correctly? (apps/ai-copilot: knowledge file, mode prompts, local wiki RAG, live wiki fetch, safety rails, endpoint answer sources) and **(B) the project's AI/search anchors** — can external AI and search engines discover it correctly? (robots.txt, sitemap.xml, llms.txt, ai.txt, JSON-LD, meta/og/canonical — on apex, web3, shop).

Verdict: READY w/ gaps. The anchor layer is unusually strong for a project this size (llms.txt on all hosts, explicit AI-crawler policy, complete sitemap, valid JSON-LD on 38/39 pages). The copilot's knowledge architecture is layered and mostly accurate on core facts — but it teaches users a **tier system that contradicts its own API**, advertises a **premium on-chain-context feature that is not implemented**, and two reliability flaws in the live-wiki fetch can silently degrade or slow every answer. The bot is not "trained" in the ML sense — it is context-grounded (static knowledge + bundled RAG + hourly live wiki); that design choice is sound, the defects are in the details.

Findings: 0 Critical · 0 High · 4 Medium · 3 Low · 1 Informational (CWA-75 ... CWA-82)

1. Copilot knowledge path (verified map)

Layer	Source	Evidence
Static knowledge	apps/ai-copilot/src/context/ifr-knowledge.ts (319 lines), JSON-stringified into every mode prompt	system-prompts.ts:27-28, 88, 119, 177
Local wiki RAG	pre-built src/context/wiki-content.json (36 docs, 2,000 chars/doc, ≤8 docs/mode)	server/wiki-rag.ts:18-151
Live wiki fetch	hourly crawl of ifrunit.tech/wiki/* + landing, tag-stripped, 4,000 chars/page, ≤80k chars injected into the system role with "Always prioritize"	server/index.ts:90-156, 481-484
Surface context	per-host allowlisted app hints	server/surface-context.ts:3-8
Model	claude-haiku-4-5-20251001 via fetch (no SDK), max_tokens 500	server/index.ts:488-501
Not in chat path	RPC/Etherscan reads (standalone REST endpoints only); points backend is POST-only telemetry	index.ts:527-537, §6 table in evidence

RAG slot note: the priority lists have 7 entries and the filler always lands on agent.html as the permanent 8th doc in every mode (wiki-rag.ts:124-151) — the page carrying the CWA-73 overstatements is therefore *always* in context.

2. Knowledge completeness vs canonical facts (block 25974493)

Covered and correct (spot-checked): token address/decimals/genesis, fee split and cap, LP pair, bootstrap parameters and outcome, LP stranding (W17), governance 48h/TreasurySafe 3-of-5, CEX policy, CommitmentVault address + balance + fail-closed price path, LendingVault address + 3 offers + rates + collateral tiers, rewardBps 1500/cap 4M, LiquidityReserve 200M/50M-cap, LP Reserve 400.6M, BuilderRegistry address + 0 builders, 1,000-IFR premium gate. The core is solid.

Wrong/stale and absent → findings CWA-77/78/79 below. **Absent from the static file** (reachable only via RAG/live-wiki, if at all): points/voucher mechanics, FeeRouter state (5 bps/25 cap/371,544 stranded), lending interest split + unset protocolFeeReceiver, vesting schedule details, Treasury 0 / Community 7.9M balances, per-contributor claim amounts, the W15 guardian exception. Full matrix: `evidence/content-audit/copilot-checks.txt` + agent ledger.

3. Findings

Medium

[CWA-75] Live-wiki fetch wipes last-good cache on failure and refetches inside every chat request

- **Severity:** Medium · **Status:** Open · **Component:** `apps/ai-copilot/server/index.ts:90-156`
- **Evidence:** the outer `catch` (:148) only logs; execution then writes `liveWikiContext = results.join()` (:152) — on discovery failure `results` is empty, so the **last good context is destroyed** and the timestamp bumped (:153). The early-return (:97) requires a non-empty cache, so until the next successful crawl **every `/api/chat` call runs the full sequential fetch loop** (per-page 5 s timeouts, :130) before the Anthropic call.
- **Impact:** a transient outage of the wiki index turns into multi-second latency amplification on every chat request plus degraded answers (static knowledge only) — with no disclosure to the user. Self-reinforcing under load (CWA-12 context: cost guard is warn-only).
- **Fix direction:** only overwrite cache/timestamp on success; serve stale-if-error; move the refresh out of the request path (background timer).

[CWA-76] Live-wiki discovery follows off-origin `.html` links into the system prompt

- **Severity:** Medium · **Status:** Open · **Component:** `server/index.ts:106-123`
- **Evidence:** the discovery regex accepts **any** `href="http..."` ending in `.html` found on the wiki index page (:110-114) and injects the fetched text into the system role capped at 80k chars with "Always prioritize this context" (:483). The CWA-24 trust boundary (hourly fetch of project pages) silently extends to any third-party host the wiki index links to.
- **Impact:** today the wiki index links only in-origin (verified against the page at the pinned commit) — but a future off-origin link (compromised third party, typo to a lookalike domain) lands unfiltered in the bot's authoritative context. No content sanitization beyond tag stripping; no prompt-injection screening.
- **Fix direction:** restrict discovered URLs to the `ifruit.tech` origin; consider a hash-pinned wiki snapshot as the authority instead of live HTML (CWA-24).

[CWA-77] The copilot teaches two contradictory tier systems

- **Severity:** Medium · **Status:** Open

- **Evidence:** user-mode prompt and widget welcome teach "Bronze 1K, Silver 2.5K, Gold 5K, Platinum 10K IFR" (`system-prompts.ts:95` , `widget index.ts:358`) while the knowledge file and the live endpoints use Basic/Premium/Pro at **500/2,000/10,000** (`ifr-knowledge.ts:244-248` ; `/api/ifr/check logic index.ts:1498-1501` , `1913`). A user asking the bot "which tier do I get for 1,500 IFR" receives a different answer from chat than from the check API.
- **Impact:** in-product contradiction about the product's core gating numbers; same family as the CWA-20 wiki tier drift, but this instance is *operational* (the API enforces one table while the bot teaches another).
- **Fix direction:** one tier table, imported by prompt, knowledge file and API from a single constant.

[CWA-78] "Premium Copilot uses verified on-chain wallet/lock context" is not implemented

- **Severity:** Medium · **Status:** Open
- **Evidence:** claims at `system-prompts.ts:65` ("users who lock $\geq 1,000$ IFR unlock Premium ... based on verified on-chain lock status"), `:103` ("it can use verified wallet and lock context"), `ifr-knowledge.ts:64-66` , and the React component (`IFRCopilot.tsx:218`). The chat handler injects **no** chain state: the `x-wallet-address` header feeds only the points telemetry (`index.ts:527-537`); the RPC reads serve the standalone REST endpoints, not `/api/chat` (`:442-540`).
- **Impact:** users lock 1,000 IFR expecting the advertised personalized on-chain guidance; the feature they paid attention (and a lock) for does not exist at runtime. Distinct from CWA-53/73 (README/wiki overstatements) — this is the runtime prompt + in-product claim itself.
- **Fix direction:** either inject verified balance/lock context server-side for locked wallets (chain read at request time, cached), or remove the claim from prompts, knowledge and widget.

Low

[CWA-79] Stale/wrong static-knowledge rows (cluster)

- **Severity:** Low · **Status:** Open
- **Instances (lead-verified):** `ifr-knowledge.ts:40` supply frozen at block 25812380 (997,673,879.09) vs canonical 997,571,140.02 @ 25974493 — honestly block-pinned but aging; widget dev welcome "544 documented tests • 91% branch coverage" (`index.ts:359`) vs 644+30+36 (CWA-20 family); `ifr-knowledge.ts:156` builder "tests: 27/27" vs 30; `ifr-knowledge.ts:139` proposal ledger omits executed #13/#14 while `:188` and `system-prompts.ts:154` credit exactly those with the Buyback wiring (internal contradiction); `ifr-knowledge.ts:153` `builderRegistry.sepolia = 0x77e9...A6e7B` — the **mainnet token** address (copy-paste).
- **Fix direction:** refresh the snapshot values; single canonical proposal list; correct or delete the Sepolia field.

[CWA-80] Copilot policy/UX contradictions and latent defects

- **Severity:** Low · **Status:** Open
- **Instances:** "NO CONTRACT ADDRESSES IN RESPONSES" (`system-prompts.ts:37`) vs dev mode listing 12 addresses and instructing the model to cite them (`:137-148` , `:171`) — outcome is model-dependent; widget history cap 40 (`index.ts:397`) vs server cap 20 (`:465-468`) → users hit a hard 400 mid-conversation; copilot's `checkHealth` declared view (`index.ts:1574`) but the contract function is state-changing (LendingVault.sol:308, emits MarginCallWarning) — latent, uncalled today; `POST /api/bootstrap/vote` still accepts "finalise"/"refund" votes although the bootstrap finalized 05.06.2026 (`index.ts:1202-1226` ; extends CWA-13's unsigned- vote finding with a sunset question).

- **Fix direction:** scope the address ban to non-dev modes; align caps; correct or remove the ABI entry; sunset the vote endpoint.

[CWA-81] AI-anchor defects on the web3 host (+ minor apex items)

- **Severity:** Low · **Status:** Open
- **Evidence (live-fetched 2026-09-14, `evidence/content-audit/anchor-*.txt`):**
 - `https://web3.ifrunit.tech/llms.txt` drops the exemption qualifier: "Every transfer burns 2.5% permanently" — false as stated (apex `llms.txt` correctly says "standard transfer between non-exempt addresses").
 - `https://web3.ifrunit.tech/sitemap.xml` lists **ifrunit.tech** URLs (cross-host sitemap — ignored by search engines for URLs outside the host) and misses `wiki/liquidity.html` (38 vs 39 entries).
 - Apex `sitemap.xml` lists `https://ifrunit.tech/web3/` although that page canonicalizes to `https://web3.ifrunit.tech/` (sitemap↔canonical mismatch).
 - `lastmod` is stale on 27 of 39 sitemap entries (2026-04-07) — pages changed through September (e.g. `bootstrap/wiki` updates in June–September).
 - `docs/builder.html` is the only page without a JSON-LD block (38/39 have one; all blocks parse as valid JSON-LD — `evidence/content-audit/jsonld-check.txt`).
- **Fix direction:** sync the web3 copies with apex; per-host sitemaps with same-host URLs; regenerate `lastmod` at build time; add JSON-LD to `builder.html`.

Informational

[CWA-82] Anchor completeness notes (policy choices, not defects)

- No `llms-full.txt`, `humans.txt`, `.well-known/ai-plugin.json` on any host (all optional).
- AI-crawler policy is explicitly `Allow` for GPTBot, ClaudeBot, Google-Extended, PerplexityBot, anthropic-ai, cohere-ai on all hosts — a deliberate, consistent choice worth recording.
- Anchor-carried instances of known page claims: apex `llms.txt` repeats "48-hour timelock, no instant changes" (→ CWA-62) and "all verified on Etherscan" (→ CWA-20 family); both flow to external AI answers.
- Wiki-content defects (CWA-57...74) flow into copilot answers via the prioritized live fetch — cross-reference, not a new finding: fixing the wiki fixes the bot's second knowledge source.

4. External anchor inventory (all live-fetched 2026-09-14)

Anchor	apex ifrunit.tech	web3	shop	Notes
<code>robots.txt</code>	✓ AI-allows + sitemap ref	✓ same policy	✓ precise allow/disallow + Host + Sitemap	shop excludes <code>/api/ /b/ /p/ /r/</code> by design
<code>sitemap.xml</code>	✓ 39/39 pages complete, no orphans	△ cross-host + 38/39	✓ 5 URLs matching robots	apex <code>lastmod</code> stale on 27
<code>llms.txt</code>	✓ comprehensive, mostly chain-accurate	△ wrong burn qualifier	✓ accurate (matches code-verified copy)	no <code>llms-full.txt</code> anywhere
<code>ai.txt</code>	✓ minimal, correct pointers	✓ same	✓ (in repo public/)	—
JSON-LD	38/39 pages, all valid	✓	n/a (app shell)	<code>builder.html</code> missing

Anchor	apex ifrunit.tech	web3	shop	Notes
meta/og/twitter	present across pages	✓	✓	spot-checked
canonical	self-referential; /web3/ → web3 host	✓ correct dedup	✓	verified

Apex llms.txt verified in detail: all **14** contract addresses correct (incl. BuilderRegistry `0xdfe6636D...0EE3` vs contract-map), test counts correct (CommitmentVault 46, LendingVault 56 — counted in repo), block-pinned supply honestly labeled, "Contributor locks verified for C1/C2/C3" **chain-verified TRUE** (`getTrancheCount = 2/10/11 @ 25974493`), CEX/listing status accurate, "seller rewards are not active" correctly stated.

5. Verified strengths

- The copilot's core knowledge is accurate on everything spot-checked (§2 list) — addresses, fees, vault states, rates, governance — the layered design (curated static + bundled RAG + hourly live refresh) is the right architecture for a docs-heavy project.
- Client-side seed-phrase interceptor exists in the React component (IFRCopilot.tsx:55-65) — but see CWA-53: the served inline widget lacks it.
- The anchor layer is best-in-class for the project's size: AI-crawler policy explicit, llms.txt on all three hosts with mostly verified content, complete sitemap, valid JSON-LD, correct canonical dedup of the web3 double-hosting.
- Safety rate limits, body/message caps and CORS allowlist are implemented in code (not just prompts).

6. Limitations

- No POSTs to the live chat endpoint (series boundary: anonymous-browser GETs only) — answer quality was assessed from the knowledge path statically, not by sampling live answers.
- The bundled `wiki-content.json` snapshot was treated as build-faithful to `docs/wiki/` (spot-checked); its build timestamp is not pinned in-repo.
- Copilot live config (model, env) inferred from source + `/api/health` behavior documented in CWA-11; no authenticated probing.

7. Disclaimer

AI-readiness review of the project's own copilot and its public discovery anchors. Not an ML evaluation, not a security audit of the LLM vendor, not financial advice. All verification read-only. AI-assisted analysis; every finding was re-verified line-by-line by the lead before inclusion (`evidence/content-audit/copilot-checks.txt`, `anchor-*.txt`, `jsonld-check.txt`).